Full Title*

Subtitle†

Anonymous Author(s)

Abstract

Developing programming languages is a challenging task. Among run-time performance, security, available libraries, or the development community, tools for the language developers and users are an important factor for the success of a programming language and the productivity of its users. To ease the process of developing new programming languages, specialized frameworks have come up.

However, frameworks that focus on run-time performance of the language implementations often provide little support to create tools for said languages. This is an impediment to language adoption that can be as serious as the potential performance issues of new language implementations when compared to established ones that these frameworks are already addressing.

In this work, we propose an approach to close the tooling gap of new language implementations by making the Language Server Protocol available automatically to new language implementations on performance-focused frameworks. Furthermore, we show how our approach can utilize run-time information to provide Integrated Development Environment features rarely available for dynamic programming languages.

Keywords keyword1, keyword2, keyword3

1 Introduction

Writing programs for computers with reasonable effort requires the use of fitting programming languages. As use-cases evolve so do languages and new ones are still being developed to deal with the rising complexity of modern software systems. Developing a new programming language from scratch is a challenging task, and for a long time, making sure the performance of new implementations is competitive with existing ones has consumed a large amount of development time. To address this, frameworks such as RPython and Truffle have been created which provide good performance through just-in-time compilation, with little effort required on the part of the language developer.

However, another barrier for new languages has been tool support. Users of new languages often have to give up on existing tools and learn new ones, if there even are replacements for all the tools that exist for established languages. This is not a given, since tool development is a significant undertaking.

The Language Server Protocol (LSP) tackles the problem of tool development by connecting multiple Development Environment (DE) with the a language server, freeing language implementers from the burden of implementing their own environments or integrating with multiple existing ones. Yet it is still significant effort to provide the features a good Development Environment requires.

In this paper, we propose an approach to provide tooling for new language implementations with comparable ease and little effort on the part of the language implementer. We show that a generic language implementation framework like Truffle or RPython can do a large amount of the work required to support the Language Server Protocol in a generic fashion. Language implementers merely have to use the hooks provided by the framework in order to get an integration with any Language Server Protocol (LSP) client for free.

Another challenge for Integrated Development Environments (IDEs) is the support of features like code completion, goto-definition, or finding all references for dynamic programming languages. These features are typically implemented in IDEs using type information. For programming languages with typed symbols, like Java, the required information can be easily derived from by parsing source code. The situation is different for dynamic languages like Python, which have untyped symbols. For these languages, the type of a variable depends on the associated run-time object only. Therefore, IDEs typically try to infer types and/or make use of additional type hints in source code comments or other annotations. Good type inference engines are difficult to build and require deep language-specific knowledge. Depending on the language, there might also exist certain dynamic language constructs which prevent any kind of type inference.

Therefore, our approach also includes suggestions on how to collect and utilize run-time information, to improve certain IDE features for dynamic languages. As proof of concept, we have implemented Graal-LSP, a polyglot language server implementation for the GraalVM.

1.1 Contributions

This thesis aims at enabling language developers to create high-performance language implementations while getting IDE-like tools for these languages with minimal additional effort. Hence, language developers and users can benefit

*Title note
†Subtitle note

PL'18, January 01–03, 2018, New York, NY, USA
2018.

from the availability of these tools from early on. Furthermore, by building upon the LSP, our approach provides them the freedom to use their favorite development tools for the new languages. As IDE-like tooling support is difficult to provide for dynamic languages, this thesis further suggests an approach to collect and make use of run-time information to enhance certain LSP features. Our contributions are in particular:

- An approach to build a language-agnostic language server on top of performance-focused language implementation frameworks.
- An approach to provide IDE-like features for dynamic languages, by collecting and utilizing run-time information.
- An implementation of these approaches using the Truffle framework [30], which includes polyglot code completion and goto-definition possibilities.
- Suggestions on how to extend the current Truffle Application Programming Interface (API), to support further LSP features.

1.2 Outline

In section 2, we provide background information concerning LSP, language implementation frameworks, and further aspects helping to understand our approach and implementation. section 3 describes our approach on how to build a language-agnostic language server on top of a run-time focused language implementation framework. After that, we explain in section 4, how we have implemented the Graal-LSP prototype based on our approach. An experience report of how well existing language implementations work with Graal-LSP, is given together with limitations in section 5. section 6 discusses approaches and solutions related to our work. Finally, we conclude our work in section 7 by summarizing our approach and describing opportunities for future work.

2 Context

In this chapter we explain terms, concepts and technologies helping to understand our approach in section 3 and the implementation of Graal-LSP in section 4.

2.1 Source Code Editors

A source code editor bundles different features to support developers in their process of writing code. Typically, code editors like Vim[1], Sublime Text[2], Atom[3], or Visual Studio Code[4], focus on editing file-based programming or scripting languages. In addition to the actual text editor view, modern graphical code editors also consist of a file explorer, graphical debugger interface, and/or build-tools integration, which are often made available as optional extensions for a variety of languages. Popular IDEs like Eclipse[5], Visual Studio[6], or IntelliJ[7] also have a code editor view as main component, but focus on a richer support of one specific language by providing tailor-made, integrated tools. In the following, we will describe selected features of source code editors, which are covered by the LSP described in subsection 2.2.

Navigational Features Source code editors often provide some actions or views to navigate through source code files. Additionally to a pure textual search, there might be a hierarchical outline view of symbols of the current source file or actions to find all definitions or references of a symbol.

Informational Features Another group of features tries to visualize source code related information, e.g. by allowing to hover with the cursor over a function call, to display signature and documentation, or over a variable, to display type information. Furthermore, if the cursor is placed at a symbol in the code, all other locations of the symbol are often highlighted in the current file. For example all read and write accesses of a local variable. If a language parser detects syntax errors or a linter provides some warnings, these information are also displayed in the code editor, typically by underlining the affected source sections.

Code-modifying Features Source code editors often provide some actions to format the whole file or a code snippet according to some formatting rules. Additionally there might be a lightweight refactoring support, like the renaming of a symbol in the current file or all files in the workspace. More sophisticated refactoring tools are rather found in IDEs than source code editors.

Code Completion An important feature of source code editors and IDEs is code completion, which is typically integrated seamlessly into the textual editor view. We describe this feature in more detail in the following, because we will use it as a running example in our approach in section 3.

Code completion can be triggered either manually via keyboard shortcuts, or automatically when a user types a trigger character. For example in the Java programming language, such a trigger character would be the dot character inserted after a variable. The suggested completion items could therefore be all names of the Java methods or fields declared by the variable's type and super types. If available, a documentation is often also included in the completion item and for callable objects also a signature. This kind of code completion, where the completion items are depending on the symbol in front of the trigger character, we call in our thesis *completion of properties*. For non-object-oriented

[1] https://www.vim.org (accessed 2018-12-17)
[2] https://www.sublimetext.com (accessed 2018-12-17)
[3] https://atom.io (accessed 2018-12-17)
[4] https://code.visualstudio.com (accessed 2018-12-17)
[5] https://www.eclipse.org/ide (accessed 2018-12-17)
[6] https://visualstudio.microsoft.com (accessed 2018-12-17)
[7] https://www.jetbrains.com/idea (accessed 2018-12-17)

languages, *completion of properties* might not be supported or be limited to special language elements, which understand some kind of *property* semantic. *Completion of properties* is often a challenge for languages with untyped symbols, like Python or JavaScript, so that only some IDEs provide a proper code completion system. Typically, IDEs like PyCharm use static type inference to obtain types for Python expressions.

The other kind of code completion, which you get in editors when the caret is placed in an empty line and the user triggers the completion manually, we call *completion of globals and locals*. In this case, the completion items are depending on the surrounding scopes. With *globals* we refer to all the symbols which we can derive from global scopes of languages. With *locals* we analogously refer to all symbols which we can derive from local scopes of languages. In the Python programming language for example, globals are built-in functions or built-in classes. Locals of a function scope are function parameters or local variables. *Completion of globals and locals* might also be triggered automatically, when the user starts typing a symbol, so that matching globals and locals are suggested.

Because code completion is a core feature and is frequently auto-activated while typing, Asaduzzaman has collected requirements of code completion systems [2]. Most of them address the fact, that the completion system should not interrupt the user's flow of code writing.

2.2 Language Server Protocol

The Language Server Protocol (LSP)[8] is an approach to standardize language support for development tools, backed by multiple IT companies. It tackles the problem of language and tool developers, which need to repeatedly implement code completion, goto-definition, syntax error reporting, and other features for every new code editor, IDE or development tool. The LSP defines a *language server* and a *language client*. The server needs to be implemented only once for every language, providing the language-specific smarts. A development tool implementing a language client can make use of any language server and therefore support multiple languages with minimal effort.

At the date of writing, language servers for more than 50 languages exist[9], including almost all popular programming and scripting languages. Not all implementations support the full feature set defined in the LSP. Usually, these language server implementations support exactly one language, but the LSP allows multiple languages per server.

The LSP version 3.7.0, which is the reference for our approach and implementation of Graal-LSP, specifies requests for typical features of a source code editor: e.g. code and signature completion, navigational features (goto-definition, find all references and implementations, etc.), information on hover, or code formatting. Syntax highlight support is not covered in the protocol and still needs to be handled by tool-specific configuration files or APIs.

[8] https://microsoft.github.io/language-server-protocol (accessed 2018-12-17)
[9] https://microsoft.github.io/language-server-protocol/implementors/servers (accessed 2018-12-17)

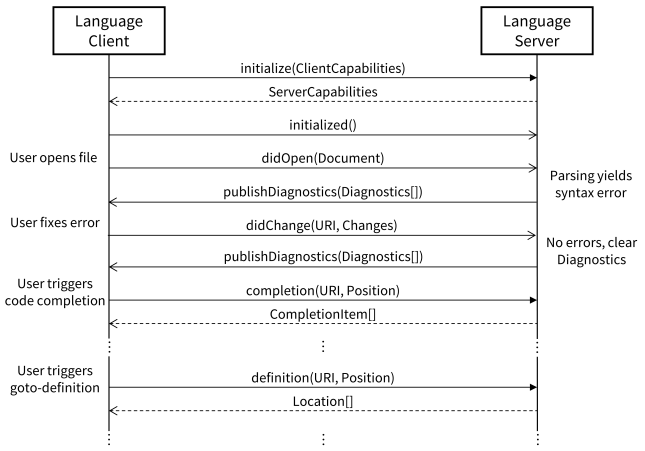


Figure 1. An exemplary communication of language client and server, including initialization, opening and changing a file, and requesting a code completion and goto-definition.

Client and Server Communication LSP client and server communicate via JSON-RPC[10] requests, responses and notifications. Source code files are referenced by Uniform Resource Identifiers (URIs). A location in the source code is encoded as line and column tuple. During initialization, client and server can exchange which features are supported.

Figure 1 shows an exemplary communication between server and client. After initialization, the client notifies the server that a file has been opened. The server parses the source code, detects a syntax error and pushes diagnostics to the client. The user fixes the displayed syntax error and the client sends these changes incrementally to the server. After parsing again on the server side, no errors occur, but the server needs to explicitly notify the client about that, so that the client can clear the currently displaying diagnostics. Then, the client sends a request for code completion at a specific source location, typically the caret position. The server obtains completion items depending on the provided context, which might also contain the trigger character, and sends the items to the client. Later, a request to find one or more definitions for a symbol at a position in the source code is sent to the server. The server responds with a list of locations, if found.

[10] https://www.jsonrpc.org/specification (accessed 2018-12-17)

LSP4J Our implementation of Graal-lsp uses LSP4J[11] as Java binding for the Language Server Protocol. This framework maps the lsp requests and notifications to methods of Java interfaces so that language server implementors do not have to deal with the json-rpc communication layer and can make use of Java's type system.

2.3 Terms and Concepts

Abstract Syntax Tree (ast) Parsers for programming languages usually transform plain text source code into an Intermediate Representation (ir) which can be processed by compilers, interpreters, or programming environment tools. An Abstract Syntax Tree (ast) is such a typical representation which stores structural information of the source code implicitly in its tree structure. Therefore, concrete syntactic elements needed to write parsable code in plain text can be omitted. In general, inner nodes of the ast represent programming language operations and leaf nodes store the values needed for these operations. Depending on the use case, an ast can store additional information in its nodes. For example the lexical source section from which a node was derived. Programming environment tools can use these information to display any kind of errors, warnings or hints directly in the plain text source code. An ast interpreter evaluates an ast by traversing its tree structure and executing the operations defined in the nodes.

Scopes and Frames A scope defines a region of a program in which a set of name bindings is valid, e.g. a local variable name. Most programming languages have a concept of a global (program wide) scope and local scopes. In a lexically scoped language, these local scopes can be obtained at compile-time and are for example defined by the nesting of modules, functions or blocks in the source code. In a dynamically scoped language, these local scopes depend on the call stack, i.e. the nesting of function calls. Today, most programming languages are lexically scoped. At run-time, frames store the actual values of the name bindings of visible scopes.

2.4 Language Implementation Frameworks

Language Implementation Frameworks help a language developer in the process of creating a new language. The feature set varies immensely, depending on the purpose of the framework. Language workbenches[12], like Spoofax [14], Xtext [6], or MPS [28], are used to develop and compose Domain Specific Languages (dsls) [7] together with a corresponding ide with minimal effort. The focus is therefore on the rich support for tools to easily and efficiently create, compose and work with dsls. Run-time performance is not the main focus. On the other side of the spectrum, there are frameworks like Truffle [30] or RPython [1], which focus on implementing high-performance programming languages. Tooling for the developed languages and especially ide-like tooling, which you get by language workbenches, has not been a focus yet. However, Truffle is an exception, due to its smart integration of an api for building tools.

Truffle and GraalVM Truffle [30] is a language implementation framework written in Java. The language developer uses the framework to write an ast interpreter, which is able to optimize itself during interpretation [33]. To achieve that, Truffle uses run-time type information to rewrite ast nodes. Callable elements of a language, e.g. functions or methods, are modeled as root nodes, which describe how these callables can be executed. A node can have a source section attribute which encodes the range in the source code file it relates to. How source code is parsed and the ast is structured leaves Truffle in the responsibility of the language implementation.

Truffle has built-in support for developing language-agnostic tools by providing an instrumentation api [24]. It is designed to work well with the optimization techniques for language implementations so that instrumentation can be done with nearly zero-overhead [23]. Instruments can be installed, like languages, by putting them on the Java class path to load them via Java's Service Provider Interface (spi) . Instruments can register and unregister listeners dynamically for different events to observe behavior of the language interpreters. For example, a listener can be notified when a new source or source section is loaded, a statement is going to be executed or an expression will return a value. It is also possible to inject behavior into the language interpreters by attaching node factories. It can be used for example, to alter the current execution flow and let a function return a different value when called under certain conditions. This instrumentation enables building powerful tools like debuggers or profilers which need to be implemented only once and can then be used for all Truffle languages. To control which ast nodes of a language are available for instrumentation, language implementors need to mark nodes as instrumentable. A tagging mechanism allows adding meta-information, like "this node represents a statement in my language", which can be used by instruments to filter for specific nodes. For example, a debugger can facilitate these information to implement stepping. The tools available at the date of writing are mostly run-time tools like debuggers, dynamic profilers and language extensions [23].

Cross-language interoperability is another feature of the Truffle framework [10]. Language implementations can be hosted in a multi-language run-time and communicate via language-independent messages. Foreign language objects can be accessed directly without marshalling.

To get high-performance language implementations, Truffle uses the language-independent just-in-time compiler named

[11] https://github.com/eclipse/lsp4j, version 0.4.1 (accessed 2018-12-17)

[12] A term created and popularized by Martin Fowler in 2005, https://martinfowler.com/articles/languageWorkbench.html (accessed 2018-12-17)

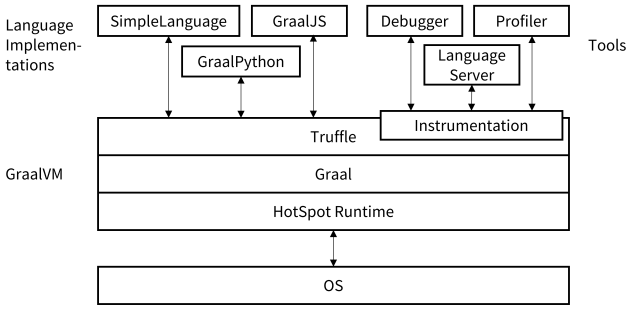


Figure 2. Truffle languages and tools hosted by the Truffle framework on top of the GraalVM.

Graal. It has its roots in the research project of the Maxine VM [29]. Among other optimizations, Graal applies partial evaluation to compile stable AST subtrees into optimized machine code [8, 31]. These optimizations do not stop at language boundaries, so that the access of foreign language objects can also be optimized.

GraalVM[13] is the umbrella term for a HotSpot VM which is bundled with the Truffle language implementation and instrumentation framework and the Graal compiler, building a high-performance polyglot VM for applications written in a Truffle-implemented language [32]. Figure 2 visualizes that architecture.

PyPy and RPython PyPy [20] is an alternative implementation of the Python [25] language. It uses a tracing Just-in-time (JIT) compiler [3] so that many Python programs can be executed faster than using the default Virtual Machine (VM) implementation *CPython*.

RPython [1] is a translation and support framework for developing dynamic programming languages. The RPython language is a restricted subset of Python. It disallows some of the dynamic features of Python to apply static analysis. Thus, source code written in RPython can be translated to C by deriving low-level C types from Python data types. Language implementors use the framework to write a bytecode interpreter with hints for the RPython tool chain, so that the interpreter can be optimized by PyPy's tracing JIT compiler.

RPython provides no explicit API for tools as described for the Truffle framework. The only notable example of a run-time tool is VMProf, a profiler to understand and resolve performance bottlenecks. The language developer registers entry points for the profiler by annotating functions with certain Python decorators and by implementing a callback to get string representations of the guest language objects.

[13] https://www.graalvm.org (accessed 2018-12-17)

Listing 1. An example Python code snippet showing a function with two parameters and two local variables to explain completion of globals and locals.

```
1  def foo(param1, param2):
2      x = 42
3
4      y = param1 + param2
5
6      print(x * y)
```

3 Approach

In section 1 we described the problem, that run-time performance-focused language implementation frameworks lack IDE-like tooling support for the developed languages. Therefore, we suggest to build a language-agnostic language server on top of these kind of frameworks to start closing the tooling gap. Because most of the currently implemented languages of frameworks like RPython or Truffle are dynamic and the support of features like code completion is especially difficult for these kind of languages, we focus on dynamic and dynamically typed language implementations in our approach.

Typically, a language server is developed for one specific language only and the server implementation can exploit any kind of language-specific information. To implement a language-agnostic language server, we have to build upon the abstract concepts and APIs of language implementation frameworks, which apply to all languages developed in these frameworks. In the following, we use the example of code completion to describe our approach. First of all, we show which kind of data is needed to respond to the *completion* request of LSP. Then, we suggest how language implementation frameworks can obtain the needed information in a language-agnostic way.

3.1 Completion of Globals and Locals

As described in subsection 2.1, there are two kinds of code completions we have to distinguish. We start with the Python code snippet in Listing 1 to explain how to support code completion of globals and locals.

Imagine, the listed Python code snippet is the content of a file which was opened in an LSP-supporting source code editor and the language client implementation has already reported the opening of the file using the *didOpen* notification to our language server. This notification includes the content and the URI of the file. Now, a *completion* request is sent to the server for line 3 at character offset 4. This means, the completion is requested in the empty line between the assignments of the two local variables `x` and `y`. We would expect a completion result to include items for all Python globals (e.g. built-in functions) and the locals `param1`, `param2`, and `x`. `y` should not be included, because a value is assigned to it not until line 4. Besides a display name, a LSP

completion item can optionally contain a completion kind (e.g. variable, function, or class), a detail text for signature or type information, and a documentation. To provide these items, we suggest the following steps.

Parsing Source Code into a Language-agnostic IR First of all we have to parse the source code into an IR of the framework. Hence, we can reuse the existing parser, which the language developer has to provide anyway. This parser will typically be syntax-error intolerant, so that we can support code completion only if the source code is parsable.

Mapping Line and Column to the IR Then, we assume that selected elements of the IR contain references to their lexical representations in the source code, or at least that there is a process to obtain these references afterwards[14]. These information allow us to search for an IR element which maps best to the position of the completion request. Which one that should be in particular depends on the IR and the framework. We therefore assume, that language implementations can expose further meta-information in a language-agnostic way through the framework, so that we can determine relevant language elements, like read- and write-accesses to variables, calls to callable elements, or definitions.

Deriving Lexical Scopes From the IR-element As a next step, we assume that the language implementation framework has a concept of scopes for the language implementations. For now, we are interested in lexical scopes only. Typically, there are some language built-ins which are always available, independent of the current context. They can be grouped in a global scope. The IR-element, matching the completion request position, can be used to derive local scopes together with symbols valid in these scopes[15]. To determine scope-building language elements in a language-agnostic way, we rely again on the aforementioned exposing of meta-information through the framework.

Building Completion Items From Scope Symbols Finally, we create the completion items based on the symbols of the scopes. Depending on the meta-information available for each symbol, we can add a completion kind, signature information for callables, or a documentation text. Due to the dynamic nature of our targeted languages, these information will be typically not available at parse-time.

[14]If we take an AST as IR, which is directly parsed from source code, relevant nodes could store start- and end-offsets of characters obtained from the current parser context. If the framework's IR is not created from source code, but from another IR, for example language-agnostic LLVM bitcode, we assume that references to the textual source code can still be obtained from debug information stored in the IR.

[15]Taking an AST as IR example again, local scopes can be obtained by traversing the AST, starting at the node, which matches the completion request position, up to the root.

3.2 Obtaining and Using Run-time Information

A detailed code completion for globals, locals, and properties of language elements is often implemented in IDEs using type information. For programming languages with typed symbols, like Java, the required information can be easily obtained by parsing source code. The situation for dynamic languages with untyped symbols, like Python, is different, because the type depends on the associated run-time object only. There are language-specific approaches to statically infer types of variables for such languages, but these approaches cannot be applied to certain dynamic language features. Furthermore, we are not aware of a performance-focused language implementation framework, which supports static type inference in a language-agnostic way for their implemented dynamic languages. The RPython framework uses type inference internally to translate the source code written in the RPython language to a more efficient, statically typed representation. But this is only possible, because RPython is a subset of Python where some dynamic features are not supported. The static type inference for RPython can therefore not be easily applied to arbitrary dynamic languages developed with RPython.

Due to the described limitations of static type inference, we focus in our approach on how to enhance code completion for locals using run-time information. This approach will also build the basis for *completion of properties* described in subsection 3.3.

Approach and Challenges The basic idea, to get run-time values for locals at a specific position in the source code, is to execute that source code while observing the execution to collect run-time information at the desired position. This approach implies some preconditions. Language implementation frameworks need to provide a possibility to execute a program until a specific source code position is reached and then to inspect the current run-time state. This is already possible, if the framework provides a debugger for the developed languages. If present, we can reuse the same approach as used by the debugger to halt at a breakpoint and obtain local symbols and their corresponding run-time values from the active frame. To create the actual completion items, the symbols can be used as display names. Details, completion item kind, and documentation can be determined from the run-time values if the framework supports accessing these meta-information by querying the language implementations.

Executing arbitrary source code requires to deal with unexpected and unwanted side-effects during execution, the possibility of a long running or non-terminating execution, or the fact that the desired position might not be reached by the execution flow.

Unreachable Source Sections Depending on the source code, there might be source sections which are unreachable

by the normal execution flow of the current source code file. In our opinion, this is the normal case for file-based programming languages. You typically split your program's source code into multiple files, having a limited set of files, which define entry points, which then execute elements of the other files.

One of these entry points are unit tests. Programmers typically write them to ensure the desired behavior of their programs and avoid regression of bugs. Therefore, unit tests should cover most parts of the source code. Furthermore, most unit tests have a short execution time and are supposed to be free of unwanted side-effects – or at least have a clean-up routine included. The systematic execution of unit tests is therefore a promising approach to get run-time information for code completion enhancement. This idea is inspired by *type harvesting* [11], a practical approach to obtaining typing information in dynamic programming languages by exploiting unit tests. We only have to provide programmers a way to tell our language server how to execute the tests. This could be done via a separate configuration file or via a special comment in the first or last lines of a source code file, which encodes a path to a runnable script.

Unit tests might not be available, especially if you start writing a small script or do not follow Test-Driven Development (TDD) principles. Therefore, we suggest as fallback to try executing the current source code, in which the code completion was triggered, if no path to an entry point is defined.

Unsaved Changes Using the described approach, we have to care about unsaved changes. If we execute unit tests, we have to ensure that they do not import and execute files from the file system, if our server has newer, but yet unsaved versions. The LSP specifies no request sent from server to client to trigger saving files and we do not want to write the changes to the file system from server-side, without asking the user. Therefore, we suggest to redirect file accesses of files with unsaved changes to (virtual) files which contain the changes. This could be realized by installing a custom file system, if the framework supports that. Otherwise, an enhanced code completion using run-time objects is only possible after saving all changes actively at client-side.

Sandboxing the Execution Environment Users would not expect that the source code, they are currently working on, will be executed when they trigger code completion. Therefore, we suggest to create a sandboxed execution environment, when the execution of arbitrary source code is triggered implicitly. To create such a sandbox, the framework needs to provide an abstraction layer, which allows to observe and restrict all I/O of the implemented languages. This requirement is a superset of the one described to handle unsaved changes. If installing a custom file system is supported, the most radical approach would be to forbid all I/O in our sandboxed environment. Unfortunately, our approach of using run-time information for code completion would thereby be limited to theoretical examples only. Therefore, we suggest to allow at least some operations. For standard I/O, we suggest to allow any kind of input, but redirect or disallow all output. For file I/O, we suggest to simulate a read-only file system or, if supported, install a custom copy-on-write file system, which copies each file before modification to a sandbox and which redirects further read and writes to that copies. For network I/O, we cannot make any assumptions concerning side-effects and suggest therefore to disallow all network I/O. The same is required for calls to native libraries.

The situation is different, if users explicitly allow side-effects. This might be the case if they have defined an entry point, like a unit testing script, which performs only known and uncritical side-effects. Therefore, we suggest to provide a configuration option, to tell our language server, if sandboxing shall be deactivated when collection run-time information for code completion.

Long Running or Non-terminating Executions If the completion request takes to long to return a result, users might become impatient and distracted. Unfortunately, the execution of arbitrary source code might take an unpredictable amount of time or might not finish at all, for example due to an endless loop or blocking operations. Furthermore, the LSP does not support responding completion results incrementally. Therefore, the first thing we suggest is to explicitly stop execution when the execution flow has reached the desired position in the source code and we have collected the needed run-time objects to build our completion items. Additionally, we suggest to add a general timeout to ensure an upper bound of execution time. In case of a timeout, no run-time information can be provided to enhance our code completion, but an endless execution can be avoided.

Caching Run-time Information Executing a script or unit tests for every completion request might yield no results due to long running executions and timeouts. Therefore, we suggest to collect run-time information for the whole source code once and cache these information for later completion requests. This collection run should be triggered manually by the users, e.g. by using custom commands, which the server can register during LSP initialization. These commands can then be made accessible at client-side via the *codeLense* request. This way, users can trigger the renewal of the cache manually, when they considered it reasonable.

As a language-agnostic data structure for the cache, we suggest a mapping from lexical source sections of IR-elements to collected run-time objects. To get a look-up key when a completion request arrives, we can again use the approach of finding an IR-element near the completion request's position, to get the IR-element's lexical source section. If the look-up succeeds, we can use the cached information to create the completion items.

Listing 2. A Python code snippet showing a property completion example.

```
1  class Natural:
2      n = 2
3
4      def next(self):
5          r = self.n
6          self.n = self.n + 1
7          return r
8
9  natural = Natural()
10 natural.
```

The challenge of using such a lexical mapping is, that every time a *didChange* notification arrives, one or more sections of the source code will change. Depending on the change, the mappings in the cache might become invalid, but others might still be useful. We assume, that most edits will affect only mappings of the current local scope and in that scope only the edited line and following, so that all other mappings might be migrated. If the mapping keys reference source positions only via line number and character offset in that line, the migration could be done by simply adjusting the line number, if the changes add or remove one or more lines. Of course, we can only assume that the migrated mappings are still valid. The user has to trigger a new collection run manually when, for example, the execution flow is changed or variables are renamed.

3.3 Completion of Properties

The second kind of code completion we want to support is the *completion of properties*.

Listing 2 shows a Python code snippet, containing a class `Natural` with an instance variable `n` and a method `next`. In the last line, we assume that the dot was the last character inserted and a code completion request has been triggered. We expect, that a completion result includes `n` and `next` as properties of the object referenced by the symbol `natural`. Other inherited properties, which all Python objects have, might also be included.

To support this kind of code completion, we can build on the approach of obtaining and using run-time information described in the previous section. But there are some additional challenges which we have to solve. First of all, we have to detect that an object property completion shall be triggered instead of a completion of globals and locals. Second, the source code will typically contain a syntax error, when this kind of completion request is sent. Third, we have to identify the IR-element for which the completion of properties is desired. Fourth, we have to obtain the actual object associated with that IR-element, which might be available only at run-time. Finally, we need to get the object's properties to create the completion items.

Detecting an Object Property Completion As prerequisite for completion of properties, we need to be able to query a language implementation for its completion trigger characters. When a completion request arrives, we can then distinguish the two kinds of completion by comparing one or multiple characters directly in front of the completion request position with the trigger characters of the current language. If there is a match, we try a property completion. To support auto-activation of code completion, the trigger characters need to be sent to the client as part of the *initialize* response.

Finding the Desired Node To find an IR-element in front of the completion trigger character, the source code needs to be parsable. We assume, that this is not always the case when a completion is triggered, because an inserted completion character might cause a syntax error. Therefore, we suggest to revert the last textual change which was sent from the client via the *didChange* notification and then try to parse the reverted source code again. Translated to our Python example in Listing 2, the source code shown is no valid Python source code because of the last dot. Removing this dot makes the code parsable again.

Now, we can search for a matching IR-element in front of the removed completion trigger character position, by checking its lexical source section. However, there might be multiple IR-elements whose lexical source sections end before the removed trigger character position. Furthermore, we are only interested in an IR-element which represents an expression, because we want to obtain its result. Hence, we assume again, that language implementations can expose meta-information of IR-elements through the framework, so that we can filter for example for expressions only.

Obtaining the Actual Properties To get the expression result, we need to execute the corresponding IR-element. In our Python example, we would evaluate an IR-element representing a variable read for the symbol `natural`. We can use the already described approach for obtaining run-time information for locals, to execute the source code until the desired IR-element is reached. Then, the framework needs to provide an API, so that we can intercept the result of the IR-element execution. Having the run-time object, we need the support of the framework to query the language implementation for properties of that object, together with further meta-information of these properties, to add details, kind, and documentation for each completion item.

We suggest some optimizations to avoid executing source code for every property completion request. First of all we

can optimize property completion for literals, if the framework supports identifying IR-elements as literal representations. The literal value is typically encoded in or referenced by the IR-element. If we can access this value, we can directly query for its properties.

If the IR-element is not representing a literal, we can check as next step if there are already cached run-time information available. If the framework and the language implementation support in-line evaluation, we can execute the IR-element in the context of the cached information.

If there are no cached run-time information available, we need to execute unit tests or the current source code until the IR-element is reached, to get the required run-time object.

If there is still no result, because the source position of the completion request might not be reachable or a timeout occurred, we can use a last fallback. We suggest to evaluate the IR-element in the global scope. This will at least provide a property completion for globals and context-independent expressions.

3.4 Transferring the Approach to Other Requests

The approach of supporting code completion, described in the previous sections, includes parts, which can be used to support further LSP requests. Table 1 shows the selected LSP requests, for which we will describe the approach transfer in the following. We have chosen these requests, because they build together with code completion the essential feature set of an IDE for dynamic languages in our opinion and work well with our approach for code completion.

In particular, we will highlight how to support *hover*, *definition*, and *references* requests for callables and local variables, the *signatureHelp* request for callables, and the *documentHighlight* and *rename* requests for local variables only. Furthermore, we will highlight, how to support the *symbol* and *documentSymbol* requests for any kind of symbol declaration. As already described for code completion, we rely on meta-information exposed by the language implementation framework. These information allow us to search for certain kinds of IR-elements representing a call to a callable or an access to a variable at the requested position, or to retrieve all kind of symbol declarations in a source code file.

Document and Workspace Symbols The *documentSymbol* and *symbol* response items consist of a label, range, symbol kind, and container name. To provide these information, we assume that the language implementations can expose these information through the framework.

First of all, we need to collect all IR-elements for a source code file, which define a symbol. Having them, label, range and symbol kind could be derived from the IR-element and it's lexical source section. The container name could be derived from the names of the surrounding lexical scopes. To support the workspace-wide *symbol* request, we can build the the union of *documentSymbol* requests for all files in the workspace.

Signature Help and Goto-definition for Callables The *signatureHelp* request is auto-activated at client-side, like code completion, if the server includes trigger characters for signature help in the *initialize* response. A *signatureHelp* response consists of a list of signatures, because there might be different variants, if the language implementation supports function overloading. Furthermore, the response includes two indices, telling which signature and which parameter should be shown in detail in the UI. Every signature in the list consists of a label, a list of parameters, and an optional documentation. A parameter also consists of label and documentation.

To collect all the needed information, we suggest the following steps.

1. Find an IR-element representing a call to a callable at the requested position
2. Execute the source code until the IR-element is reached
3. Intercept the callable run-time object represented by the IR-element
4. Query the callable for label, parameters and documentation

The first to third steps equal those for code completion of properties. A challenge is, that the signature help should be displayed from the time, the trigger character is inserted, until the user has completed the signature. During that process, the source code might contain syntax errors, so that we have no IR-elements. Returning an empty response will end the displayed signature help. Therefore, this request might be supported only partial, depending on the particular language syntax.

Having obtained a callable object, the last step implies, that we can query the framework for information about that callable. Among all parameter names and their documentation, we need a formatted, language-specific representation of signature to create the label. If this is not supported by the framework, but we can get at least access to a list of parameter names, we suggest to use a Python-like syntax `callableName(param1, param2, paramN)` as signature label.

The approach to support the *definition* request for callables is quite similar. Only in the last step, we do not have to query the framework for signature information, but for the callable's definition location in the source code. Furthermore, we can apply a static search as fallback, if the dynamic approach does not yield a result. We can use the same approach as for the *smybol* request, to find all definitions of callables in the workspace and filter those matching the identifier of the callable at the requested position. For non-unique identifiers, this might yield many results, which are not related to the desired definition location. But in our opinion, it is better

Table 1. Selected LSP features and their descriptions.

Request/Notification	Description
documentSymbol	Request for symbols of a specific source file (function names, variable names, module names, etc.).
symbol	Like documentSymbol, but for all files in the client workspace.
signatureHelp	Request to find signature information (e.g. parameter names and details for a function) at caret position. Is triggered via registered trigger characters.
hover	Request for information at caret-hovering position.
definition	Goto-definition request for functions, variables, etc.
references	Request to find all references of a symbol.
documentHighlight	Request to highlight all ranges related to a symbol (e.g. variable reads and writes).
rename	Request to rename a symbol workspace-wide.

to provide the users a list of potential matches instead of providing nothing.

Document Highlight, Rename and Find All References The *documentHighlight*, *rename* and the *references* responses consist of a list of ranges in the source code. For the *documentHighlight* response, each range can additionally have an associated kind (`Text`, `Read`, or `Write`), and for the *rename* response, each range needs an associated file URI and the new text.

To support these requests for local variables, first of all a IR-element representing a read- or write-access of a variable at the requested position needs to be found. Having one, we suggest to search in the surrounding lexical scopes for all other IR-elements representing an access to variables with the same identifier. The required range can be obtained from the lexical source section of the IR-element. For the *documentHighlight* response, the kind can be determined from the access kind of the variable. For the *rename* response, the URI is the one of the current file and the new text is already provided in the request.

Supporting *references* also for callables workspace-wide, we could use the same static fallback approach as described for the *definition* request. Due to potentially found false-positives, we do not suggest to also apply this static approach to the code-modifying *rename* request.

Hover Information We can support the *hover* request for variables and callables, if there are cached run-time information available. If we have found an IR-element representing a variable or callable at the requested position, we can perform an in-line evaluation as described for code completion. Using the evaluation result, we can build the hover response, by querying the framework for information about run-time objects as described for code completion. The hover response consists of plaintext or markup content, so that we can put it together depending on the available information.

4 Implementation

In this chapter, we describe how we have implemented our prototype Graal-lsp, based on the approach described in section 3. Graal-lsp uses Truffle as underlying language implementation framework and exploits Truffle's instrumentation environment to access the required information in a language-agnostic way. We also implemented a Truffle API extension to support more LSP features. As LSP Java wrapper we used LSP4J. Transportation for JSON-RPC is implemented using Java's TCP sockets. In the following sections, we start by describing how we have implemented common features required for most LSP requests and then explain in detail the implementation of selected LSP requests in Graal-lsp.

4.1 Prerequisites to Answer LSP Requests with Truffle

Implementing a language server on top of the Truffle framework with the requirements described in our approach, especially the sandboxing, has implications on our project structure, dependencies, and the bootstrapping process. Furthermore, there are common prerequisites to answer most LSP requests. These are parsing source code, mapping the requested position to source sections of AST nodes, and collecting and utilizing run-time information.

4.1.1 Threads and Context Activation

Every operation performed on the Truffle instrumentation environment requires, that the current Thread has entered a Polyglot `Context`. Because some Truffle language implementations do not support multi-threaded access, we decided to have only one worker Thread, which performs operations on the Truffle instrumentation environment. Our launcher ensures, that this Thread explicitly enters a `Context` instance at start-up once. We use a `java.util.concurrent.ExecutorService` instance to submit tasks to that worker Thread. Incoming

LSP requests are accepted by a Thread of the LSP4J framework. To avoid blocking this Thread, it only wraps a request into a task and submits it to the executor service. A separate Thread picked from a dynamic pool will await the processing results of the context-entered worker Thread using a `Future`. It will send the prepared response to the client and handle occurred errors by sending diagnostics in addition.

4.1.2 Language Server Initialization

The first request sent from the client to the server is the *initialize* request. To support auto-activation of code completion and signature help, the response has to include trigger characters. A language-specific language server implementation can provide these characters statically, because the set of supported languages is well known. In our language-agnostic implementation, where Truffle languages can be installed dynamically, we have to collect these characters for all available languages during server initialization. Using the Truffle instrumentation environment, we can get a list of all registered languages, but there is no API to obtain trigger characters for code completion and signature help from a language. Therefore, we have extended the classes `TruffleInstrument.Env` and `TruffleLanguage` with methods to retrieve these characters.

4.1.3 Parsing Source Code

To parse source code in Truffle, we need to build a `Source` object, containing the language ID and a reference to the actual source code, typically via a file. Then, we can us the instrumentation environment to call `parse(Source source, String... argumentNames)`. In-line parsing is also supported via the method `parseInline(Source source, Node node, MaterializedFrame frame)`.

The LSP requests *didOpen* and *didChange* both contain an URI pointing to the concerning source file. The *didOpen* request also contains the whole file content and a language ID. Some LSP clients send a media type as language ID. Therefore, we test if the ID belongs to a registered language via the instrumentation environment. If not found, we try to match it with the media types defined by each language.

In our implementation we configured the *didChange* request to contain only incremental changes. Therefore, we have to store the editor content together with the language ID at server-side and apply all incoming changes to keep track of the client-side state. To do so, we created the class `TextDocumentSurrogate`. An instance is created for each *didOpen* request, to store all data corresponding to a source file at client-side.

The *initialize* request can contain a workspace URI. If present, we parse all files in the workspace folder and its subfolders once, which match one of the media types of the installed Truffle languages.

In case of a parsing error, we can notify the client using the *publishDiagnostics* notification.

Diagnostics Truffle languages typically throw a `TruffleException` in case of a parsing error. The parsers are not designed to be syntax error tolerant and will stop parsing at the first syntax error. A `TruffleException` can store an AST node as location where the exception occurred. If available, we map the node's source section to line and column of the original source code and send a *publishDiagnostics* notification to the client. If the location is not available, we use the first column in the first line as reference for the notification. The error message needed for the notification is also obtained from the `TruffleException`.

4.1.4 Mapping Requested Source Position to AST Node

Most LSP requests are context-sensitive and include therefore line and column as reference. Line numbers are zero-based and a newline character at the end of a file counts as separate line. Truffle's `Source` objects use one-based line numbers and do not count an empty line at the end. Therefore we have to convert the request parameters line and column to match the Truffle sources.

As described in our approach, we have to map line and column of the LSP requests to IR-elements of a language, to use them for further querying and processing. The Truffle instrumentation environment can for example obtain lexical scopes from their AST nodes. Therefore, we are either interested in an AST node having a source section which contains the requested position (e.g. *hover*, *definition*, and other requests), or in a node having a source section which is near the requested position (*completion of globals and locals*). To search for these nodes, we can use the Truffle instrumentation environment to access all already loaded source sections and their corresponding nodes by attaching a `LoadSourceSectionListener` with a filter for the current `Source` and the `includeExistingSourceSections` parameter set to `true`. Our implemented listener collects the node with the shortest source section containing the requested position as `containingNode`, the one with the shortest source section ending before the requested position as `previousNode`, and the one with the longest source section beginning after the requested position as `nextNode`.

This selection process requires that language implementations support a consistent source section hierarchy. That is, if a node has a source section and is a child of another node, then the source section of the child is fully contained in all source sections of its direct parent or indirect parents (ignoring parents without source sections). If the assumption holds, we can take the `containingNode` as best match for further processing for requests like *hover*, *definition*, or *references*. For *completion of globals and locals*, the selection process is more complicated and will be explained in subsubsection 4.2.1.

As another prerequisite, we assume that source code is eagerly parsed when calling `parse(Source source, String...`

argumentNames) and not lazily when the code is going to be executed. Otherwise, no nodes and source sections will be available via the instrumentation environment.

4.1.5 Obtaining Run-time Information

In subsection 3.2 we explained an approach to provide a richer code completion for locals and support further LSP requests by observing the executing of source code. We describe in the following how we have implemented most of that approach in Graal-lsp.

Observing the Execution The instrumentation environment provides the method `attachExecutionEventListener(SourceSectionFilter, ExecutionEventListener)` to observe executions of the current `Engine`. The `ExecutionEventListener` will be notified immediately before and after an instrumented node is executed. The `SourceSectionFilter` allows to filter for which source sections or node types the listener shall be activated.

Long Running or Non-terminating Executions If we observe the execution using an `ExecutionEventListener`, we can stop when we have collected required run-time information. Therefore, we simply throw a special `ThreadDeath` exception, containing the collected run-time object. This exception is also used by other Truffle tools to stop or unwind the current execution. We catch this exception in our language server implementation, to retrieve the run-time object. In our approach, we have also suggested to cancel the execution in case of a timeout or when we receive a certain notification from the client. Due to time constraints, we have not implemented this feature yet. However, this could be easily achieved with a timer and regular checks through an execution listener by throwing a `ThreadDeath` exception when the timer has expired or a cancel-notification has been received.

Unreachable Source Sections As described in our approach, we need to define an entry point, e.g. a path to a script which triggers unit tests, to cover a maximum of source code. For now, we have implemented to search for the string `"RUN_SCRIPT_PATH:"` in the first line of the source code and interpret the rest of the line as file path. We create a `Source` object for that file and parse it using the instrumentation environment's `parse(Source source, String... argumentNames)` method. The return value is a `CallTarget` which we can execute. As further proposed in the approach, we fall back to executing the current source code, in which the request was triggered, if no `"RUN_SCRIPT_PATH:"` string is found.

Unsaved Changes To handle unsaved changes, we have suggested in our approach to install a custom file system, which intercepts read accesses to files and delegates them to a virtual file system which includes the changes. By using the Graal Software Development Kit (SDK) as described in **??**, we can create `Context` instances and can use the method `Context.Builder.fileSystem(FileSystem fileSystem)` to install our own file system. We have implemented such a file system in the class `LSPFileSystem`. It holds a callback to our language server, to delegate read accesses, if the file is known by the language server.

Sandboxing the Execution Environment To support basic sandboxing of the file system, we made our `LSPFileSystem` read-only. At the date of writing, Truffle supports restrictions on file I/O, host language access, and native calls only. Therefore, network and standard I/O is still unrestricted in our implementation. Global execution state of languages is represented in the Truffle framework by `Context` objects. After executing source code to collect run-time information, the global state will certainly be modified. Therefore, sandboxing global state can be simply realized by creating new `Context` instances.

Reusing Context Objects Unfortunately, creating new `Context` objects might be quite expensive, depending on the language implementation. Therefore, we do not want to create them for every incoming LSP request. Especially code completion requests need to be answered fast enough, so that users are not interrupted in their flow of writing source code. Although the global state might be modified by source code executions, we decided to reuse `Context` objects until the user manually triggers a full collection run to cache run-time information.

Caching Run-time Information To cache run-time information, we use the instrumentation environment to attach an `ExecutionEventNodeFactory`. We let the factory create special `ExecutionEventNode` instances, which capture a snapshot of the passed frame at the first call of `onEnter(VirtualFrame frame)`. As data structure, we use a `Map` associating `MutableSourceSectio` with `CoverageData`. Every `TextDocumentSurrogate` instance holds a reference to such a `Map`. A `MutableSourceSection` is a light-weight references to a node's source section by storing only line, and column for start and end. A `CoverageData` instance stores a `MaterializedFrame` and the `ExecutionEventNode` which captured the data.

The collection of run-time information can be started manually by the user by activating a command, which we provide via the *codeLense* request in the first source code line.

To avoid invalidation of all our mappings, when a *didChange* notification arrives, we use a simple migration strategy for our `MutableSourceSection` instances.

1. If a cached section's end line is smaller than the start line of the change, we do nothing.
2. If a cached section's start line is greater than the end line of the change, we check if the change adds or removes lines. If so, we adjust start and end line of the cached section by that difference.

Listing 3. An example Python code snippet to explain how to find the best matching node having a source section near the requested completion position.

```
1 def foo(param1, param2):
2     x = 42
3
4     y = param1 + param2
5
6
7 def bar():
8     return 42
```

3. If the range of the change is fully included in the cached section, we adjust only the end line of the cached section by the difference of added or removed lines.
4. In all other cases where start or end lines of cached section and change overlap, we assume that the mapping is not valid anymore.

This way, we can migrate cached run-time information mappings for simple changes. But we can only assume that the adjusted mappings are still valid. The user has to trigger a new collection run manually, if a change alters execution flow, names of variables, or types of variables.

4.2 Implementing LSP Request Handlers

In this section we will explain how we have utilized the Truffle instrumentation environment to support selected LSP requests. For or most LSP requests, we have implemented a separate Java class, bundling common features in their super class `AbstractRequestHandler`.

4.2.1 Completion of Globals and Locals

To implement the code completion approach described in subsection 3.1, we can mostly use already existing methods from the Truffle instrumentation environment. The method `findTopScopes(String languageId)` provides global scopes of a language and the method `findLocalScopes(Node node, Frame frame)` provides the local scopes. To use the latter, we have to find a node representing a source section near the requested completion position.

Finding the Nearest Node We explain the difficulty of finding that node by using the following Python code snippet.

Listing 3 shows a function definition `foo`, containing two parameters and two local variables in its scope, and a function `bar` without locals. Imagine, that a completion request is triggered in line[16] 3 at character offset 4. As already mentioned, Truffle cannot derive scopes directly from a source code position, but from an AST node. The difficulty is, that the derived scopes will contain variables valid *at* that node. This means translated to our Python example, that if we use the node representing `y = param1 + param2` in line 4 to derive local scopes from, the scope of function `foo` will contain variable `x` but not variable `y`. One could say, that we get scopes with all locals valid before the node's source section starts. Imagine another code completion is requested in line 5 at character offset 4. The problem is now, that there is no node which will provide us with a scope containing the variable `y`, although we are still in the function scope of `foo` and the completion is triggered one line after a value was assigned to `y`. The IntelliJ's code completion system avoids such problems for example by inserting a dummy structure element at the caret position if there is none. As Truffle instrument, we cannot modify an AST in such a way.

However, as described in subsubsection 4.1.4, we can use the Truffle instrumentation environment to find a `containingNode`, `nextNode`, and `previousNode` in the neighborhood, based on the requested position. We use the following best-effort algorithm to decide which node will most likely deliver the expected locals for the completion items.

1. If we cannot find a node having a source section which contains the requested position, we are not in a local scope. No completion of locals necessary. (Example: line 6, offset 0)
2. If there is one which contains the requested position, we pick `nextNode` if it is also an (indirect) child of `containingNode`. (Example: line 3, offset 4; picking the node representing `y = param1 + param2`)
3. If there is no `nextNode`, we pick `previousNode` if it is also an (indirect) child of `containingNode`. (Example: line 5, offset 4; picking the node representing `y = param1 + param2`)
4. If there is still no match, we use `containingNode`. (Example: line 4, offset 10; picking the node representing `param1`)

Querying Properties of Scope Values Calling `findTopScopes(String languageId)` or `findLocalScopes(Node node, Frame frame)` returns a collection of `Scope` objects. A `Scope` can be lexical and dynamic. `Scope.getVariables()` returns a `TruffleObject`, which can be queried in a language-agnostic way via sending messages. Using a `KEYS` and a `READ` message, we can obtain key-value pairs, where the key is the name of the declared variable and the value is again a `TruffleObject` representing the assigned value.

For locals, we can only obtain lexical scopes when calling `findLocalScopes(Node node, Frame frame)` with `frame=null`. The returned local `Scope` objects might contain declared variable names, but no values will be available. Therefore, we create completion items for locals by using the variable name as display name, the scope name as detail text, and `CompletionItemKind.VARIABLE` as default completion item kind.

[16] LSP uses zero-based line numbers. Truffle's `Source` objects reference lines one-based. For simplicity, we will always use one-based line numbers in our examples.

For globals, we can therefore create completion items with more details, because a value will typically be available. We can query the value, which is a `TruffleObject` again, for meta-information by sending the following messages. To determine a completion item kind, we use `IS_INSTANTIABLE` and `IS_EXECUTABLE`. If the first returns `true`, we use `CompletionItemKind.CLASS`. If the second returns `true`, we use `CompletionItemKind.FUNCTION`. Default is again `CompletionItemKind.VARIABLE`. To add the name of the value's meta-object to the detail text of the completion item, we use the Truffle instrumentation environment to call `findMetaObject(LanguageInfo language, Object value)`.

We added two more custom messages (`GetDocumentation` and `GetSignature`) to support querying a `TruffleObject` for documentation and signature of callables. If a language implementation does not support `GetDocumentation`, we try if we can get the source code of the callable definition as documentation for the completion item, using the method `findSourceLocation(LanguageInfo language, Object value)` of the instrumentation environment.

Using Cached Run-time Information If cached run-time information are available when a completion for globals and locals is triggered, we use the nearest node's source section to search for a cached `CoverageData` object. If found, we can call `findLocalScopes(Node node, Frame frame)` with the cached frame. Now, the resulting `Scope` objects will also give access to declared local variables. We can use the same approach as described for globals to get details and documentation for the completion items.

4.2.2 Completion of Properties

Following the approach described in subsection 3.3, we have implemented the *completion of properties* in our Graal-lsp prototype.

Detecting an Object Property Completion To distinguish a completion request for globals and locals from one for object properties, we can use the API extension already suggested in subsubsection 4.1.2, which allows us to ask a Truffle language for its completion trigger characters. Because the *initialize* response contains the trigger characters of all installed languages, the client will automatically trigger completion requests for all these characters. Therefore we have to check every request, if the trigger character belongs to the current language. If not, we have to skip the request.

Including the completion trigger character in the request is an optional feature of the client. Furthermore, the character is only included if the completion was triggered automatically. To support code completion of properties also when triggered manually, we assume, that we can distinguish the two kinds of completion by comparing the character directly in front of the requested position with the registered trigger characters of the current language. If there is a match, we try an object property completion.

Finding the Desired Node As described in our approach, the source code will most likely not be parsable when a completion of properties is triggered. We assume that the last inserted text snippet is the completion trigger character, which caused the syntax error. Therefore, we simply store the last change for every `TextDocumentSurrogate` object to replace the last inserted text snippet with an empty string in the source code.

Having parsable source code, we can search for a node directly in front of the removed text snippet. We use the same approach as for finding the nearest node for a completion of locals and globals, but filter for nodes tagged with `ExpressionTag` only. If we find a `containingNode` whose source section ends directly in front of the removed text snippet, we take it to obtain its corresponding run-time object. In the following we refer to this node as `propertiesRequestedNode`.

Strategies to Obtain Properties The next step we have to do is getting the run-time object represented by the `propertiesRequestedNode` to finally query that object for its properties. We obtain run-time information as described in subsubsection 4.1.5, but we have implemented further optimizations for this kind of completion.

First of all we can optimize property completion for literal nodes in Truffle. For instrumentable nodes, we can use the method `getNodeObject()` to get a `TruffleObject`. We assume, that it supports reading a value for the key `"literal"`. If supported, we can use the read value to create completion items for its properties. If the value is a `TruffleObject`, we obtain its properties as described for globals and locals in subsubsection 4.2.1. But the returned value might also be a primitive Java type, which needs to be boxed into a language-specific `TruffleObject`, so that we can query it for properties. Therefore, we added the method `boxPrimitive(String languageId, Object primitive)` to the instrumentation environment API.

If the literal-approach returns no results, we check for cached run-time information. To make use of them, we apply in-line evaluation, which requires that the Truffle language implementation supports in-line parsing. First of all, we traverse the AST, starting at the root node of our `propertiesRequestedNode`, to find a node in the neighborhood, which has a mapping to `CoverageData` in our cache. To call `parseInline(Source source, Node node, MaterializedFrame frame)`, we create a `Source` object by using the source section text of our `propertiesRequestedNode`. As second parameter, we use the `propertiesRequestedNode` itself. The `frame` is taken from the `CoverageData` object. If in-line parsing is successful, the returned result is an `ExecutableNode` instance. To execute that node, it needs a parent. Therefore, we temporarily insert it as child into the `ExecutionEventNode` which we was stored in the `CoverageData` object during the run-time collection run. The properties of

the execution result are queried as described for literals in the previous paragraph.

If there are no cached information yet, we start executing source code and observing the execution until it reaches our `propertiesRequestedNode`. We interrupt the execution at this point as described in subsubsection 4.1.5, to retrieve the evaluation result of the `propertiesRequestedNode`.

As last fallback, we try to parse and execute the source section text of the `propertiesRequestedNode` in the global scope of the language.

4.2.3 Further Requests

Signature Help As proposed in our approach in subsection 3.4, we have implemented signature help support similar to the completion of properties. We added the method `getSignatureHelpTriggerCharacters(String languageId)` to the instrumentation environment, to retrieve the needed trigger characters. In contrast to the completion of properties implementation, we assume that the source code is parsable after the trigger character is inserted, because many language clients support smart character insertion, so that for example the insertion of the widely used trigger character '(' is automatically amended to '()'. Furthermore, we do not search for a node having a source section in front of the trigger character, but for a node at the requested position, tagged with a `CallTag`.

If found, we execute the source code as for completion of properties, but we listen for slightly different events, because we want to obtain the callable object itself, instead of the return value of the called callable. Therefore, we have implemented the `onInputValue(EventContext context, VirtualFrame frame, EventContext inputContext, int inputIndex, Object inputValue)` method of the listener, which provides intermediate results of expressions. In our case, we can intercept the callable object before it is called.

Having the callable, we send the added custom message `GetSignature` to retrieve a `TruffleObject` representing a list of signature parameters. As described in our approach, the language needs to provide a formatted string representation of the signature which we can use as label for the response. We assume therefore, that the retrieved `TruffleObject` has a `format()` method, which we can call by sending an `INVOKE` message.

To set the active parameter index of the response, we simply use the number of provided arguments. We assume, that the `CallTag`-annotated node can tell us that number. Therefore, we call `getNodeObject()`, which should return a `TruffleObject` supporting the reading of a value for the key `"numberOfArguments"`.

Hover Information Our implementation of Graal-lsp supports hover information for variables and callables, if cached run-time information are available. We use the already described approach to find a node corresponding to the requested position. If found, we try two strategies to get a run-time object represented by that node.

First of all, we test if the text of the node's source section matches an identifier of one of the frame slots. We can access these slots by calling `frame.getFrameDescriptor().getSlots()`. If found, we use the value associated to that frame slot. If not found, we in-line evaluate the text of the node's source section, as described for the completion of properties in subsubsection 4.2.2.

If one of the strategies yields a result, we use again the same implementation as described for the completion of properties, to query the object for meta-information, signature, and documentation.

Document and Workspace Symbols To find symbol declarations in the current file, we added a new tag `DeclarationTag`. We use the instrumentation environment's `LoadSourceSectionListener` to search for all tagged nodes in the current `Source` object. Then, we assume that a call to `getNodeObject()` returns a `TruffleObject` supporting the look-up for the keys `"name"`, `"kind"`, and `"container"`. We can map these information together with the node's source section to the *documentSymbol* response items.

If a Truffle languages does not support `DeclarationTag`, we apply a simple fallback. We use the instrumentation environment to search for all nodes tagged with the already existing `RootTag` in the current `Source` object. This way, we get at least names and source sections of declarations of callables, to fill the *documentSymbol* response items partially.

To search for symbols workspace-wide, we use the same implementation as described for the *documentSymbol* request, but we omit the filter for the current `Source` object.

Document Highlight We have implemented *documentHighlight* support in Graal-lsp for variables. To make nodes representing a variable access detectable, we added two new tags, `ReadVariableTag` and `WriteVariableTag`. We assume, that nodes tagged with one of them further implement `getNodeObject()`, returning a `TruffleObject` which supports the look-up key `"name"`. If such a tagged node can be found at the requested position, we use the instrumentation environment to find local scopes based on that node. Then, we traverse the root node of each scope by calling `accept(NodeVisitor nodeVisitor)` and search for children tagged with either `ReadVariableTag` or `WriteVariableTag`. We take those nodes, having the same `"name"` as our initial node and build the response using their source sections.

Goto-definition and Finding All References The *definition* request has been implemented for callables workspace-wide and for variables source-local. We search for a node at the requested position, which is tagged with `CallTag`, `ReadVariableTag`,

or `WriteVariableTag`. If found, we use different approaches for callables and variables.

For nodes of callables, we obtain the corresponding run-time object as described for the completion of properties in subsubsection 4.2.2. Then, we use the method `findSourceLocation(Language language, Object value)` of the instrumentation environment, to find the source section of the callable.

If the run-time approach yields no results, we use a static fallback search. We take the implementation of the *symbol* request to collect all symbol declarations in the workspace and filter for those matching the identifier of our callable node. This static search is also used to implement the *references* request for callables. As already mentioned in our approach, this might yield false-positive matches. But as the response consists of a list of locations, we can provide all of them and let the users choose from these suggestions.

For variables, we assume that the first write of a local variable is its definition location. This way, we can reuse the implementation described for the *documentHighlight* request, to find all other occurrences of a local variable and pick the first write access as definition location. To implement the *references* request for variables, we also reuse the *documentHighlight* implementation, to find all occurrences of a local variable.

5 Experience Report

In section 4, we have shown how Graal-lsp was implemented. To support all the described lsp features, language implementations need to implement existing Truffle apis and our proposed extensions. For some languages, this might be done with very low implementation effort, but for others, some features might require fundamental changes in their implementations. In this chapter, we show by means of the dynamic Truffle languages GraalPython, SimpleLanguage, FastR, and GraalSqueak, that a basic support of some lsp features can be achieved with minimal implementation effort. Furthermore, we explain, why certain lsp features cannot be easily supported in some language implementations. Finally, we describe limitations of our approach and implementation of Graal-lsp.

5.1 Comparison of Truffle Language Implementations

SimpleLanguage (sl)[17] is Truffle's exemplary language implementation, to showcase Truffle features and help understanding its ideas and concepts. SimpleLanguage (sl) has first class functions. The type system only consists of primitive types and a single `object` type. Objects are implemented as simple key-value stores. As showcase implementation, it implements most of the needed Truffle apis so that, after implementing our proposed extensions with minimal effort, sl supports nearly all Graal-lsp features.

[17] https://github.com/graalvm/simplelanguage (accessed 2018-12-17)

```
3                [Graal]
4                Error(s) parsing script:
5  function m -- line 6 col 11: no viable alternative at input '+;'
6      a = 3+;
7  }
```

Figure 3. Diagnostics used to display a syntax error in sl.

GraalPython[18] is an early-stage experimental implementation of Python. It does not support many libraries yet, but is mature enough for our use cases. Together with the GraalPython team, we implemented the required Truffle apis, so that GraalPython is the language implementation which supports the most Graal-lsp features now.

FastR[19] is a high-performance implementation of the R[20] programming language used for statistical computing and graphics. Its goal is to become a drop-in replacement for the reference implementation GNU-R. Due to time constraints, we have only implemented support for completion trigger characters yet, so that the FastR implementation is nearly unchanged. Therefore, not many Graal-lsp features are supported.

GraalSqueak[21] [16] is an early-stage experimental Squeak/Smalltalk [13] implementation. In contrast to the other selected languages, it is an image-based, graphical live-programming environment. Furthermore, it uses the ast-based Truffle framework to implement a bytecode interpreter. Therefore, it is difficult to map concepts of GraalSqueak to the file-based lsp protocol. Nonetheless, we have chosen it to show that at least some features can be supported with reasonable effort and to discuss limitations of our approach. We run GraalSqueak in headless mode to load an image and get access to its run-time objects. For now, we treat every opened `*.st` file as an isolated Squeak/Smalltalk workspace.

Table 2 gives an overview over the status quo of implemented Truffle apis of these languages. Futhermore, Table 3 adds a broad overview over the resulting Graal-lsp features supported by these languages. In the following, we will highlight, how well some of these features are already supported by showing and describing screenshots of the usage in a language client. Therefore, we have built a small extension in Visual Studio Code, which connects to an already running Graal-lsp instance.

Diagnostics All languages except FastR fail parsing at the first syntax error, so that only one error per file can be sent to the client. Figure 3 displays how such a diagnostics notification is visualized in sl. FastR does not throw a `TruffleException` in case of a syntax error, so that Graal-lsp cannot report any syntax errors for FastR.

[18] https://github.com/graalvm/graalpython (accessed 2018-12-17)
[19] https://github.com/oracle/fastr (accessed 2018-12-17)
[20] https://www.r-project.org (accessed 2018-12-17)
[21] https://github.com/hpi-swa-lab/graalsqueak (accessed 2018-12-17)

Table 2. An overview over selected Truffle languages and their current support of existing Truffle APIs and our proposed extensions. Adjusting and extending these language implementations for GRAAL-LSP is still work in progress, especially for FastR and GraalSqueak.

Truffle API (existing and extensions)	SL	GraalPython	FastR	GraalSqueak
CallTag	✓	✓	✓	✗
StatementTag	✓	✓	✓	✓
ExpressionTag	✓	✓	✗	✗
findTopScopes()	✓	✓	✓	✓
findLocalScopes()	✓	✓	✓[r]	✗
findMetaObject()	✓[n]	✓	✓[n]	✓
findSourceLocation()	✓	✓	✓	✗[?]
parse() eager AST creation	✓	✓	✓	✗
parseInline()	✗	✓	✓	✗
getCompletionTriggerCharacters()	✓	✓	✓	✓
getSignatureHelpTriggerCharacters()	✗	✓	✗	✗
getNodeObject() for literal nodes	✓	✓	✗	✗
boxPrimitive()	✗	✓	✗	✗
GetSignature custom message	✗	✓	✗	✗
GetDocumentation custom message	✗	✓	✗	✗
DeclarationTag, Read/WriteVariableTag	✓	✓	✗	✗

[r] Run-time scopes only
[n] Names of meta-objects only
[?] Unclear how to implement

Table 3. An overview over selected Truffle languages and their support of GRAAL-LSP features. If a feature is not supported, it will either depend on not yet implemented Truffle APIs as described in Table 2, or on missing information due to a run-time focused language implementation.

LSP feature	SL	GraalPython	FastR	GraalSqueak
Diagnostics	✓	✓	✗	✓
Completion (globals)	✓	✓	✓	✓
Completion (locals)	✓	✓	✓[*]	✗
Completion (properties)	✓[*]	✓[*]	✗	✗
Signature Help	✗	✓[*]	✗	✗
Hover	✓[*]	✓[*]	(✓)[*]	✗
Symbols	✓	✓	(✓)	✗
Goto-definition (callables)	✓[*]	✓[*]	✓[*]	✗[?]
Goto-definition (local variables)	✓	✓	✗	✗
References (static)	✓	✓	✗	✗[?]
Highlighting variables	✓	✓	✗	✗

[*] Requires run-time information
() Larger limitations
[?] Unclear how to implement

Code Completion Figure 4 shows a code completion for globals and locals in GraalPython. Completion items have different icons for classes, methods, and variables in the list of suggestions. For the selected item of a built-in class, a string representation of the corresponding meta-object is displayed, together with a documentation, if available. Figure 5 shows

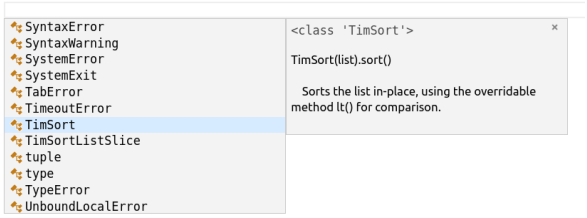


Figure 4. Code completion of globals and locals in GraalPython.

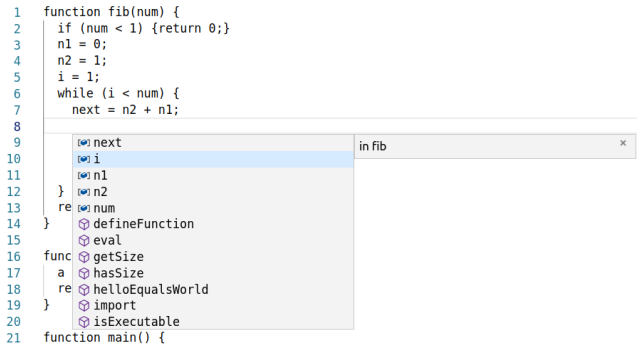


Figure 5. Code completion of globals and locals in SL.

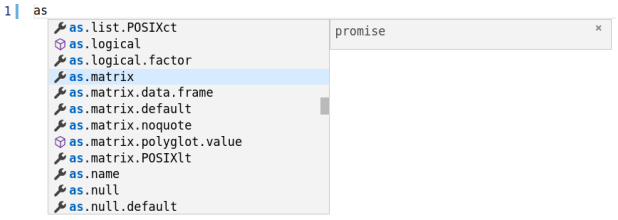


Figure 6. Code completion of globals in FastR.

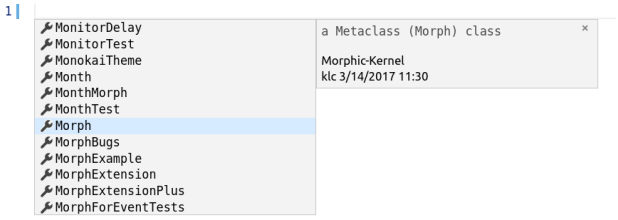


Figure 7. Code completion of globals in GraalSqueak.

a code completion in SL. Variables of lexical local scopes are listed first, followed by built-in functions. For the selected local variable, the scope name is displayed. Figure 6 shows a completion of globals in FastR, with an already filtered list of globals containing the string `"as"`. The completion of locals in FastR is supported only if run-time information, i.e. a frame, is available. Figure 7 shows globals of the Smalltalk dictionary for GraalSqueak.

The code completion of properties is supported to different extents in these languages. Run-time information are always required due to the dynamic nature of the languages. Therefore, the source code needs to be parsable after removing the trigger character. For SL, GRAAL-LSP supports the completion of properties, which are the keys of the underlying key-value store. For GraalPython, properties of objects can also be those inherited from their type hierarchy including fields and methods of their classes. For FastR and GraalSqueak, we were not able to provide a completion of properties yet, due to the missing support of `ExpressionTag` annotations for their AST nodes. But also for languages supporting the `ExpressionTag`, the applicability of our approach depends on the design of their AST. In GraalPython, triggering a completion of properties for the code snippet `anObject.nestedObject`, by inserting a dot at the end, will yield completion items for all properties of the object referenced by `anObject.nestedObject`, because the nearest node tagged with an `ExpressionTag` is the one representing the whole source section. In SL however, inserting a dot before the semicolon in `anObject.nestedObject;` to trigger code completion, will yield no results, because the nearest node tagged with an `ExpressionTag` is the `SLStringLiteralNode` instance representing `nestedObject` and not the node representing `anObject.nestedObject`. Therefore, our approach tries to derive completion items from the String `"nestedObject"`, which is the execution result of the `SLStringLiteralNode` instance. Similar cases might occur also in other language implementations, because Truffle leaves the AST design to the language implementors, so that GRAAL-LSP can only be implemented as language-agnostic best-effort approach.

Using GraalPython and SL, we want to highlight, that Truffle supports polyglot programming and we can therefore provide completion of properties across language boundaries. Figure 8 shows Python code evaluating a SL code snippet which creates and returns a SL object. In the last line, we get completions of properties for the foreign SL object, which would not be possible in SL code, as described in the previous paragraph.

The code completion request for properties is triggered fairly often in comparison to the other run-time information requiring requests. Unfortunately, the performance depends on the one hand on the execution time of the source code, which can be reduced by caching run-time information, but on the other hand it depends on the creation time of Truffle language contexts. We create a new one for each execution, which is quite cheap for SL, but for GraalPython, it takes a noticeable time. Reusing those contexts improves the performance, but introduces new problems, which we will explain in detail in subsection 5.2.

Signature Help Signature help is only supported with limitations for GraalPython yet. GRAAL-LSP can provide a formatted signature string as shown in Figure 9, but when starting

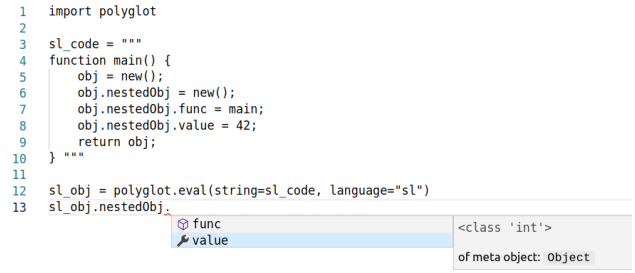


Figure 8. Completion of properties for a foreign SL object in GraalPython source code.

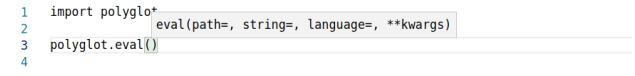


Figure 9. Signature help for GraalPython.

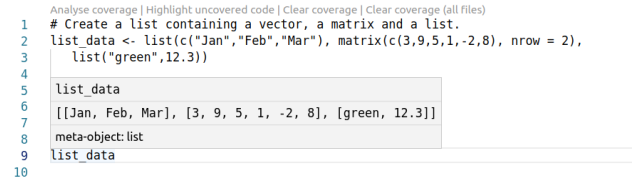


Figure 10. Displaying information on mouse hover for FastR source code using cached run-time information.

to fill in the signature, the displayed hint will be cancelled as soon as a syntax error is reported. For example when typing `"string="`, to define a keyword argument. Afterwards, the signature help request is not triggered again. The other languages do not support the `GetSignature` custom message yet. Implementing this message for example in SL might not be possible without changes to the language, because SL stores function arguments internally like any other local variable of the function, so that there is no knowledge left at run-time to obtain the signature of a function. This is one example of many, where languages need to store additional parse-time information, to support more LSP features.

Hover Showing information on mouse hover is supported for variables and expressions of SL and GraalPython yet. For FastR, only variables can be accessed via their corresponding frame slot, to provide hover information. For all languages, cached run-time information need to be available. A collection run can be triggered using the *codeLense* named `"Analyse coverage"`, as shown above the first line of code in Figure 10. The displayed hover information include a string representation of the variable's value and the meta-object.

Symbols As soon as a language eagerly creates AST nodes when the source code is parsed, Graal-lsp can support the request for symbols by providing the names of all `RootNode` instances in the current document. If the `DeclarationTag` is used to annotate nodes, we can access further information to set a fitting symbol type and container name as shown for GraalPython in Figure 11. For FastR however, taking the names of all `RootNode` instances is not useful yet, because there is one for each line with the name `<repl wrapper>`.

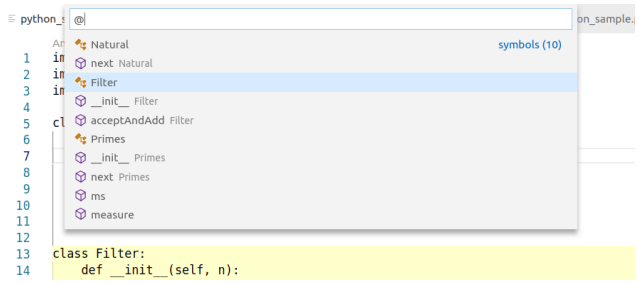


Figure 11. Symbol overview over the current GraalPython document. The selected entry's source section is highlighted in the editor.

Goto-definition and References Our goto-definition and find-all-references implementations depend on the supported tags of the Truffle language. The selected languages, except GraalSqueak, use the `CallTag` to annotate nodes representing a call, so that goto-definition can be supported using run-time information. This works also across language boundaries, when using Truffle's polyglot API. Find-all-references uses a static symbol search yet, so that it will work as soon as the code is parsable. To avoid too much false-positive matches, we disabled the search in internal sources of the language by default. For GraalSqueak, which is not file-based, we do not know yet, how a useful implementation of navigational requests might look like in general.

Variable Highlighting The support for highlighting of variable accesses depends on the usage of `ReadVariableTag` and `WriteVariableTag`. Figure 12 shows how it looks like for GraalPython. As for SL, the node's source section, representing a variable write access, does not only cover the variable name, but also the right-hand side of the assignment. This might be fine for the highlighting use case, but it makes it more difficult to implement for example the LSP *rename* request for local variables, because we would have to locate the variable's identifier in the source section by ourselves.

5.2 Limitations of Graal-lsp

Due to time constraints and limitations of the Truffle framework, we were unable to fully implement the approach described in section 3. We have not yet implemented a timeout or cancellation support for requests, to avoid an endless execution of source code when collecting run-time information. The suggested sandboxing of the execution engine is limited to file I/O for now, because Truffle does not support restriction of other I/O kinds yet. Furthermore, the sandbox

```
16  class Filter:
17      def __init__(self, n):
18          self.number = n
19          self.next = None
20          self.last = self
21
22      def acceptAndAdd(self, n):
23          filter = self
24          sqrt = math.sqrt(n)
25          while True:
26              if n % filter.number == 0:
27                  return False
28              if filter.number > sqrt:
29                  break
30              filter = filter.next
31
```

Figure 12. Highlighting of variable accesses in GraalPython. The source sections of write accesses cover the whole assignment and are highlighted in a blueish shade.

```
1   class ClassA:
2       n = 1
3
4   class ClassB:
5       nestedObj = ClassA()
6
7   obj = ClassB()
8   obj.nes
9           nestedObj                    <class 'ClassA'>
10
11  obj.nestedObj = {'x': 3}
```

```
6
7   obj = ClassB()
8   obj.nes
9           nestedObj                    <class 'dict'>
10
11  obj.nestedObj = {'x': 3}
```

Figure 13. Code completion of properties without and with cached run-time information in GraalPython. In the upper part of the image, the completion is shown as expected, because the completion items were obtained from the run-time state at line 8 by executing the source code up to that line. In the lower part, run-time information have been collected for the whole source code and the completion items were obtained from the frame snapshot captured in line 7, whose stored `obj` instance is no deep copy and therefore includes the modification in line 11.

is always activated when source code is executed, because we have not yet implemented a possibility to deactivate sandboxing when the user classifies the occurring side-effects as uncritical. Therefore, all features of Graal-lsp, which require collecting run-time information, are not supported yet, if the executed source code requires write access to the file system. For GraalSqueak, this is a serious limitation, because it continuously stores actions and source code modifications of the current image in a separate `*.changes` file and wants therefore to open this file with write permissions during start-up. This fails at the moment due to our read-only file system, so that the whole image is not initialized correctly and we are unable to retrieve documentations for classes for example yet.

Caching Run-time Information and Contexts As described in subsubsection 4.1.5, we introduced caching of run-time information and reusing of Truffle language contexts, to improve performance especially for code completion. Unfortunately, this implies some new limitations. To cache run-time information, we create a snapshot of the current `MaterializedFrame` instance at a specific source section. This snapshot is not a deep frame copy, so that only primitive values are actually copied. If fields of referenced objects are changed later, the frame snapshot will still reference these mutated objects. Figure 13 illustrates the problem using a GraalPython code snippet.

Another problem of caching frame snapshots is that we hold references to arbitrary run-time objects which can therefore not be garbage collected. Depending on the size of the objects, the Java process might run out of memory, although this has not yet happened in our experiments.

Using cached frames for in-line evaluation, to improve the performance of lsp responses, has another limitation. The executed source code must rely on data of the stored frame snapshot only. There is for example no call stack available and the language context storing the global run-time state might be different than it was at the moment when the snapshot was taken.

As already mentioned, we also reuse language contexts to avoid their expensive creation each time source code has to be executed. Unfortunately, this leads to similar global state problems as described for stored frame snapshots. For example in GraalPython, a module import is done only once per context. If the module's source code is changed and we reuse a context which has already imported that module, the changes will be ignored in further executing runs. Furthermore, after renaming functions, the old ones will still be suggested via code completion.

With regard to the numerous limitations which arise due to our current implementation of caching run-time information, we have to admit, that this part needs further improvements. It works well for context independent scripts, but is not yet mature enough for lager software projects.

A Polyglot Language Server The Truffle framework allows hosting language implementations in a multi-language runtime environment. We have already explained, that Graal-lsp supports polyglot code completion, but with some limitations. A language server is typically developed for exactly one language. During initialization, we have to tell the language client the set of completion and signature help trigger characters to support auto-activation. If the installed languages have different trigger characters, like a dot for GraalPython, a $ for FastR, and a space for GraalSqueak, a completion might be triggered in unwanted situations and might cancel other requests. FastR for example allows to use dots in variable identifiers, as can be seen in the already mentioned Figure 6. Typing a dot in FastR code while having GraalPython installed, will trigger a code completion request. At server-side, we can detect, that the dot is no trigger character of FastR and return no completion items, but the current request, which might be a previous completion

request of FastR, will be cancelled. Furthermore, Visual Studio Code displays a default, token-based code completion, if Graal-lsp returns no results. This is quite confusing for the language users, because a language's code completion behavior might change depending on the set of installed languages.

Another limitation occurs for the code completion of properties of foreign objects, if the foreign languages have different syntaxes, as already described for FastR and GraalPython. The suggested properties are formatted in the syntax of the foreign language and might therefore yield a syntax or runtime error in the current language. It lies therefore in the responsibility of the developer to deal with mismatching syntaxes and transform suggested code elements, which do not match the language's syntax, to a generic, foreign message send using the polyglot API of their current language.

5.3 Limitations of Our Approach

There are further limitations of our approach, independent of the current implementation of Graal-lsp, which we describe in the following.

Language Server Protocol A noticeable shortcoming of the LSP is the lack of being able to provide hints for syntax highlighting from server-side. Syntax highlighting helps to read and understand source code, especially if it is an unfamiliar language. Although most editors support the usage of editor-independent formats like TextMate grammar files[22], it is an additional effort for the language developers to provide and maintain these files for multiple editors. However, a support for semantic coloring is planned for a future version of LSP, which might be exploited also for syntax highlighting.

As the LSP is an abstraction over a set of typical IDE features and we add another layer of abstraction for GraalVM languages, there will most likely be language-specific IDEs, which can provide a more tailor-made and suitable tooling support than we can provide with Graal-lsp.

Reusing Existing Language Parsers We proposed in our approach to reuse the existing language parsers of performance-focused language implementations. As these parsers typically stop parsing on the first syntax error, we cannot report more than one syntax error per file to the client. Furthermore, all LSP features rely on the availability of AST nodes, to query for static information or to execute them to collect run-time information. Therefore, the source code needs to be parsable. However, when writing source code, there are situations were programmers edit the code in different areas in parallel, so that there might be multiple parts containing syntax errors. Reverting only the last change would not help in these cases to get the source code into a parsable state, so that most LSP features would not work.

Executing Source Code Numerous features of our approach rely on run-time information, which we collect by executing source code. However, this implies, that our approach does not work well if we cannot provide these information. For example, if the sandboxing prevents some kind of I/O access, the source section of interest is not covered by the current execution, or a timeout triggers an exception. Therefore, the quality of our approach and the overall user experience might vary greatly depending on the source code and the available run-time information.

6 Related Work

Our approach focuses on enabling performance-focused language implementation frameworks to provide their implemented dynamic languages with IDE-like tooling support with minimal effort. In this chapter, we describe related approaches and solutions. Furthermore, we compare them with our approach and implementation of Graal-lsp. As a major part of our approach is based on collecting and using run-time information to enhance certain LSP features, we will also discuss work related to that field.

IDE-frameworks like Eclipse [4], Visual Studio[23], or IntelliJ[24] provide extension points to develop IDE-specific plugins for new languages or can be used as language clients if they implement the LSP. But as they do not support language developers in implementing programming languages in the first place and writing IDE-specific plugins or a language server still requires a lot of work, we do not further discuss them here. Furthermore, there are lots of language- and editor-specific tooling solutions for dynamic languages, which are also not our focus.

6.1 Tools-first Language Implementation Frameworks

Language Workbenches Language workbenches are tools that support the efficient definition, reuse and composition of languages, especially DSLs [7], and their IDEs [5]. Hence, they provide the means to make the development and composition of new languages affordable, which has been proven by the usage of language workbenches and DSLs in various use cases in industry. Most language workbenches support a textual language notation, which can be edited in a free-form text editor. But there are also solutions like MPS [28], which allow projectional editing or mathematical symbols. Furthermore, syntactic services (syntax highlighting, folding, or syntactic completion), semantic services (reference resolution, refactoring, diagnostics, or semantic completion), and further IDE-like features can be provided based on the language definitions.

However, as already described in section 1, these frameworks do not focus on the performance of the language

[22] https://macromates.com/manual/en/language_grammars (accessed 2018-12-20)

[23] https://visualstudio.microsoft.com (accessed 2018-12-17)

[24] https://www.jetbrains.com/idea (accessed 2018-12-17)

implementations in the first place and are often bound to a limited set of IDEs. Furthermore, dynamic languages cannot make use of all the services of language workbenches, as these services typically rely on the static information derived from the language definitions.

Xtext [6] is one of the first language workbenches, which supports the LSP, and which also proposes, that it can be used to develope DSLs and full-featured programming languages. However, Xtext faces the same challenges like the other workbenches, when it comes to dynamic languages, which we try to tackle explicitly in GRAAL-LSP.

Visser et al. described an approach using the Spoofax language workbench [14] and the DynSem language [26] to systematically generate Truffle languages, i.e. AST interpreters, from DynSem specifications [27]. However, Spoofax provides plugins for the Eclipse IDE only. Supporting the LSP seems to be planned, but is not yet present. Hence, it would be worth to evaluate, if a Truffle language generated through this approach could provide enough means to support at least some GRAAL-LSP features. Nonetheless, the generated interpreters will still require manual tweaks, to benefit from all the GraalVM optimizations.

Helvetia and Squimera Two approaches, which focus on reusing existing tools of a live programming environment and the overall programming experience, are the Smalltalk-based IDEs Helvetia [18] and Squimera [17]. Helvetia enables embedding DSLs into Smalltalk, by adding hooks into the Smalltalk compiler chain. This way, the DSLs can be translated on demand into Smalltalk code, so that existing tools of the environment still work, without the need to generate and start a new IDE.

Squimera is not a language implementation framework, but it enables polyglot programming and debugging through the composition of interpreters written in RPython. With some adjustments, some of the existing Smalltalk tools have been made reusable for writing and debugging Python and Ruby code, providing a consistent programming experience across languages. Like GRAAL-LSP, Squimera utilizes the interpreters and APIs of a performance-focused language implementation framework, to retrieve data needed for the tools of the language users.

Helvetia and Squimera provide syntax highlighting and debugging support for the integrated languages, but features like code completion or goto-definition are only provided at Smalltalk level. However, the integrated languages could theoretically make use of all the tools of the Smalltalk environment. In Squimera, customized tools for polyglot programming of high-level languages have been created and are still a subject of research. This would not be possible in IDEs using GRAAL-LSP yet, as they are bound on the one hand to the features made available through the Truffle instrumentation API and on the other hand to the specification of the LSP.

In summary, language workbenches and the other described approaches can provide a richer, more customized, and domain-specific tooling support for new or existing languages than GRAAL-LSP. However, these solutions are not able to create high-performance dynamic languages together with tools for the users of the languages, supporting IDE features like code completion or goto-definition. Furthermore, the users are often bound to a single IDE, whereas GRAAL-LSP allows them to choose from a variety of LSP clients.

6.2 Language-agnostic Approaches Enabling IDE-like Features

To the best of our knowledge, there are no approaches comparable to the Language Server Protocol yet, which are supported by almost all popular source code editors and languages. However, there exist other approaches for language-agnostic tooling support, which can be seen as predecessors or influencer for LSP.

Ctags Ctags is an old but still popular tool to create language-agnostic index (tag) files for symbols found in source code of various programming languages. The tags can be consumed by source code editors to provide a goto-definition search or a simple symbol completion. Over the years different flavors of the original Ctags have come up which extended the original file format for tags. A popular variant, Exuberant Ctags[25], was originally distributed with Vim, but became a separate project, supporting over 40 programming languages. New languages can be added either by defining regular expressions to determine language symbols, or providing a parser.

Energize Energize/Cadillac [9] was an object-oriented IDE for C++ built upon Emacs in the late 1980s. It was based on a language-agnostic client-server architecture for programming language environments. Clients (compilers, editors, debuggers, etc.) communicate with a server using protocols for a certain class of tools (compiler protocol, editor protocol, debugger protocol, etc.). The server is able to persistently store objects that represent source code, language elements, compiled code objects, and annotations (objects which are links between other objects), so that each tool can access the needed information. For example, annotations could be used to link compiler diagnostics to source code and/or language elements, but also to retain dependencies to enable incremental compilation. The persistent object store was intended to store information, such as dependency analyses, for the lifetime of the software. New languages could be added, by providing compilers for these languages, which support the compiler protocol.

The LSP could be interpreted as an evolved version of selected parts of the Energize protocols, especially of editor and compiler.

[25] http://ctags.sourceforge.net (accessed 2018-12-16)

Kythe Kythe[26] is a pluggable, language-agnostic ecosystem for building tools that work with code. Kythe allows language analysis and indexing to be run as services, by defining language-agnostic protocols and data formats for representing, accessing, and querying source code information as data. It was originally developed as an internal Google project, due to the observation that most companies have projects which are created with more than three programming languages, but lack appropriate cross-language tools[27]. To support a new language in Kythe, a language compiler or interpreter needs to be extended, so that it can emit a rich AST, symbol table, and type graph in a Kythe-compliant intermediate representation. For dynamic languages however, the generated ASTs do not include the information required by Kythe, so that additional type inference engines are needed.

srclib Another project which tried to tackle similar challenges as LSP is srclib[28], a polyglot code analysis library for static analysis and code search. srclib was the foundation of Sourcegraph[29], a code search and intelligence server that helps developers to find, review, and understand code. Like LSP, srclib defined a language-agnostic format, which can be consumed by development tools. New languages can be supported by implementing a tool chain that outputs the srclib format. For languages like JavaScript, Ruby, and Python, tool chains have been built reusing existing language-specific projects like Tern, YARD[30], and Jedi. However, with the upcoming of LSP, the development of srclib was stopped and Sourcegraph also makes use of LSP now.

Like the LSP, the described tools and approaches define more or less complex protocols to decouple languages and their development tools. Supporting a new language ranges from adding only a regular expression to implementing or instrumenting complex compilers, to output language-specific information in a language-agnostic format. However, code completion is not part of these protocols, which we wanted to support in GRAAL-LSP. Furthermore, besides LSP, only Ctags is supported by a variety of source code editors and IDEs at the date of writing.

6.3 Language-specific Editor-agnostic Approaches Enabling IDE-like Features

As described in the introductory words to this chapter, there are lots of IDE- or editor-specific solutions to support code completion etc. for certain dynamic languages, which are not discussed here. But there are some language-specific solutions worth mentioning, which are designed to be editor-independent, like Jedi[31] for Python, Tern[32] for JavaScript, or Rsense[33] for Ruby. Like LSP, Tern and Rsense define a lightweight interchange format, so that editor-plugins can be created, which only have to map the requests and responses to their editor-specific API. Tern and Rsense provide a server which accepts and responds JavaScript Object Notation (JSON) messages. However, the structure of the messages is influenced by the corresponding language and is therefore more specific than the message format of LSP. Jedi provides a simple Python API only. Therefore, editor-plugins need to be able to execute Python code or support the LSP, because there already exist Python language servers using Jedi.

All the described tools mainly rely on static code analysis, so that they all face limitations due to certain dynamic features of their corresponding language. Jedi was originally intended to provide a rich code completion by executing non-dangerous code, which did not work, so that a highly recursive static analyzer has been created[34]. GRAAL-LSP is able to execute source code to some extent yet, so that it supports code completion also for dynamic features. However, as the described solutions are language-specific, they can support certain features more suitable and with more detail than a generic LSP. But this comes at another cost, because these tools cannot reuse the existing language parsers to get the required data. They had to built and maintain separate parsers, so that when the languages evolve, there might always be a gap between language elements already supported in the original parsers, which have not been implemented in the parsers of the tools yet. In contrast, GRAAL-LSP can indirectly utilize the existing language parsers through the Truffle API.

6.4 Improving IDE Features With Run-time Information

A major part of our approach focuses on collecting and utilizing run-time information to answer LSP requests. Therefore, we have a look at where and how other approaches apply run-time information to improve IDE features.

Hermion The Hermion IDE [21] collects run-time information to enhance program comprehension by improving static source code navigation and browsing in Squeak/Smalltalk. In detail, they apply a filter to the list of methods of the implementors view to display only those methods which have been called in a specific scenario. Furthermore, they display collected type information for instance variables and provide an additional reference view in the method source

[26] https://kythe.io (accessed 2018-12-16)
[27] https://bsumm.net/2012/08/11/steve-yegge-and-grok.html (accessed 2018-12-16)
[28] https://srclib.org (accessed 2018-12-16)
[29] https://sourcegraph.com (accessed 2018-12-16)
[30] https://yardoc.org (accessed 2018-12-16)
[31] https://github.com/davidhalter/jedi (accessed 2018-12-19)
[32] http://ternjs.net (accessed 2018-12-19)
[33] https://github.com/rsense/rsense (accessed 2018-12-19)
[34] https://jedi.readthedocs.io/en/latest/docs/features.html#a-little-history (accessed 2018-12-19)

code view, that shows a list of observed classes which are referenced in the current method.

Hermion builds upon partial behavioral reflection [22] to restrict the collection of run-time information to certain source code parts and detail levels. However, due to performance issues, the users of the Hermion IDE have to make a trade-off between performance and the amount of collected information. The users need to manually specify which classes shall be instrumented, to keep the cost of instrumentation and reflective operations at an acceptable level. In Graal-lsp, the set of AST nodes which shall be instrumented when collecting and caching run-time information, is implicitly defined by the LSP workspace, i.e. all parsed source code files. As we collect snapshots of frames, we increase performance and reduce the amount of collected instances by taking only the first frame which is reported for every instrumented node. As Hermion continuously observes the instrumented classes, it can collect different types for a variable, which the current Graal-lsp implementation would not be able to detect.

Enhancing Static Source Code Search in Eclipse Holmes and Notkin [12] described an approach to improve the static find-references feature of the Eclipse IDE using collected run-time information. Only executed methods are included in the search results, so that users benefit from a limited result set related to their current application. To collect the required run-time information, they use a custom AspectJ [15] aspect for the application's test harness.

Our Graal-lsp prototype does not utilize run-time information to answer the *references* request of the LSP yet. We perform a static symbol-based search in files of the current workspace, but it should be possible to enhance our implementation of the *references* request in a similar way.

PyCharm Debugger PyCharm is a feature-rich IDE for the Python programming language. The debugger of PyCharm is able to optionally collect run-time type information during a debug session[35]. These information are used to improve code completion and documentation popups during and after the debugging session. The collected type information can also be persisted as docstrings in Python source code.

Similar to our approach, the PyCharm's debugger can utilize unit tests as entry point. However, Graal-lsp does not require to run code in a special debug mode to collect run-time information and can also make use of these information to provide a precise goto-definition for callables. For the PyCharm IDE, this kind of dynamic run-time type inference is a small addition to the other mature features, which rely on static analysis, whereas for Graal-lsp, most features rely on run-time information. Like the previously described Hermion IDE, PyCharm's debugger collects types and is able to remember all type variants of variables, whereas Graal-lsp collects a frame snapshot per instrumented node. This way, Graal-lsp can provide code completion items also for properties of an object, which are not type related.

In summary, all the three described solutions collect language-specific run-time information to improve certain static aspects of existing language-specific tooling. Hermion focuses on code navigation and comprehension, the approach of Holmes and Notkin on finding all references, and PyCharm on code completion and documentation. In contrast, Graal-lsp collects run-time information in a language-agnostic way and tries to support as much IDE features as possible with these information. Graal-lsp uses static approaches rather as fallback when no run-time information can be retrieved.

The following two approaches do not directly target to improve specific IDE features, but they describe how to collect run-time information, so that other tools can make use of them.

PyAnnotate PyAnnotate[36] is a Python library to collect type information (call arguments and return types) at run-time, using a Python-specific profiling hook. After a collection run, these information can be automatically inserted as annotations or comments into the Python source code, so that other tools can make use of them.

Graal-lsp collects run-time objects to answer LSP requests only. The objects or their types are not persisted in the source code files for the later use of other tools.

Type Harvesting Haupt et al. have presented an approach called *type harvesting* to collect type information for dynamic programming languages [11]. They suggest to use these information for code generation, optimization, or program comprehension. Their solution is based on a stepwise execution of bytecode while observing the types of relevant components. The set of relevant components, which shall be observed, are called an *acre*, which have to be determined manually by the user before the harvesting is started. A *bucket* stores mappings of bytecode instructions, arguments, variables, etc., to the collected concrete types. Like in our approach, they suggest to use unit tests as entry points to start code execution.

In contrast to our approach, *type harvesting* continuously observes the stepwise execution of bytecodes, leaving the actual bytecode instruction stream untouched. However, the simulated stepwise execution is quite slow and they need to check for every step, if the current bytecode is on an acre. Graal-lsp benefits from Truffle's instrumentation and optimizations of AST nodes, so that the execution of instrumented nodes has a very small overhead.

[35] https://blog.jetbrains.com/pycharm/2013/02/dynamic-runtime-type-inference-in-pycharm-2-7 (accessed 2018-12-16)

[36] https://github.com/dropbox/pyannotate (accessed 2018-12-16)

Another difference is, that type harvesting stores specific mappings of bytecode instructions to types, whereas Graal-lsp stores lexical mappings of lines and columns to frames. Storing types only is obviously more memory efficient, but would not work for Graal-lsp, as we need the actual run-time objects to utilize the Truffle api.

7 Conclusion and Future Work

In this thesis, we described the problem of language implementation frameworks lacking ide-like tooling support for implementations of dynamic programming languages. Language developers have to use these frameworks to create and maintain programming languages with reasonable effort. However, they can mainly choose between tooling- or performance-focused frameworks. The former are typically focusing on making the creation and combination of dsls and their ides affordable. This comes at the cost of performance for these languages. Furthermore, the users are often forced to choose from a limited set of ides, which might not include their favorite development tool. In contrast, the performance-focused frameworks provide the means to build high-performance language implementations also for dynamic languages, but support at most some run-time tools.

However, providing tooling support like code completion, goto-definition, and finding all references for dynamic languages is also a challenge for the tooling-focused frameworks. They rely on static analysis, which does not work well for languages with untyped symbols and further dynamic features.

Therefore, we presented an approach which attempts to provide the means for building high-performance dynamic language implementations together with ide-like tooling support, with minimal additional implementation overhead for the language developers. The approach is based on building a language-agnostic lsp server on top of performance-focused language implementation frameworks. Using the lsp enables languages users to stick to their favorite development tools, as the protocol is supported by almost all popular source code editors and ides. Furthermore, we suggested to collect and utilize run-time information to support certain ide-features also for dynamic languages.

As proof of concept, we have implemented Graal-lsp, a language server for the GraalVM. It uses the instrumentation api of the Truffle language implementation framework, to parse and execute source code of Truffle languages and answer lsp requests. No separate parser or inference engine is required. A sandboxing of the file system avoids unwanted side-effects when executing arbitrary source code. As by-product of using the Truffle framework, Graal-lsp supports a polyglot code completion and goto-definition. Furthermore, we have suggested small api extensions for Truffle, to support more lsp features. They have been implemented in selected Truffle languages, together with other yet unsupported parts of the Truffle api. As a result, we could demonstrate, that a basic set of important lsp features can be supported for most of these languages with minimal implementation effort.

7.1 Future Work

Handling Current Limitations As future work, we first of all suggest to tackle the current limitations of Graal-lsp. The sandboxing implementation could be improved with the already named suggestions of a copy-on-write file system and a possibility to disable the sandbox if the user classifies the side-effects of the code execution as uncritical. Furthermore, supporting a timeout for requests would be useful to avoid non-terminating executions. Most lsp clients already send cancellation notifications, but Graal-lsp does not apply them yet. In our opinion, cancelling code executions could be achieved by adding an execution event listener through the instrumentation api of Truffle, checking in regular intervals if a cancellation notification has been received or a timer has been expired. In that case, a `ThreadDeath` exception can be thrown to stop the current execution.

The caching of run-time information is essential to improve the performance of certain Graal-lsp features, but the current implementation implies too many limitations yet. Therefore, a solution, which does not have to store whole frame snapshots referencing run-time objects, should be developed. As described in subsection 6.4, other approaches collect and use only the type information of the run-time objects to improve ide-like features. It needs to be seen, if a type-based approach could be used with the current Truffle api or which kind of extensions would be needed.

Supporting Further lsp Features For now, we have implemented only those lsp features, which would in our opinion be either implementable with reasonable efforts using the current Truffle api, or would bring the greatest benefit for language implementors and users. Therefore, one future work would be to implement further lsp requests (*rename*, *formatting*, *foldingRange*, *typeDefinition*, etc.) or improve those which are implemented only partially yet (e.g. *references*). However, implementing these requests might require additional Truffle api extensions and language asts containing more parse-time information.

At the moment, syntax highlighting still needs to be defined editor-specific at client-side. As semantic coloring is on the agenda of a future lsp specification release, it would be useful to investigate, if this feature might be exploited to implement syntax highlight from server-side.

Improving ide Support For Polyglot Programming As shown in subsection 5.1, we provide a basic support for cross-language code completion of properties and goto-definition for foreign callables. Because ide support for polyglot programming is a unique feature of Graal-lsp, it would be useful to improve this area with further support from the Truffle framework.

For example, smaller foreign code snippets are sometimes directly defined as string literals in the source code of another language. It would be useful to display syntax or run-time errors of the foreign language directly inside these string literals.

Furthermore, we described in subsection 5.2, that a polyglot language server implies limitations regarding trigger characters of multiple languages. It needs to be seen, if this limitation can be avoided or if it might require an adjustment of the LSP specification.

Supporting Further Truffle Languages For now, only GraalPython and SL support most of GRAAL-LSP's features, as described in subsection 5.1. Therefore, the required Truffle APIs, existing and new, need to be implemented for the other mentioned languages. Furthermore, there are Truffle languages, which we have not yet extensively tested with GRAAL-LSP, like Graal.js[37] and TruffleRuby[38]. The last time, we tested Graal.js, eager source code parsing was not supported, so that most features could not be used. Nonetheless, a support for Graal.js and TruffleRuby should be achievable with a similar low effort as for GraalPython.

With GraalSqueak, we have already started to investigate, how languages might be supported, which require an additional compilation step from their source code to the Truffle AST. For example, GraalSqueak compiles the Smalltalk source code to bytecode first, before the Truffle AST is created. LLVM-compatible languages can compile their sources to LLVM bitcode, which can then be translated into a Truffle AST by Sulong [19]. GraalSqueak includes the required compiler already, but for LLVM-compatible languages, it needs to be seen, how the compilation step can be triggered by GRAAL-LSP. Having the bitcode files, it should be possible to support GRAAL-LSP features by mapping requests triggered in the actual source code to the debug information in the LLVM bitcode files.

References

[1] Davide Ancona, Massimo Ancona, Antonio Cuni, and Nicholas D Matsakis. 2007. RPython: a Step Towards Reconciling Dynamically and Statically Typed OO Languages. In *Proceedings of the 2007 Symposium on Dynamic Languages (DLS)*. ACM, 53–64. https://doi.org/10.1145/1297081.1297091

[2] Muhammad Asaduzzaman. 2018. http://hdl.handle.net/10388/8530. *Context-Sensitive Code Completion.* phdthesis. University of Saskatchewan. http://hdl.handle.net/10388/8530

[3] Carl Friedrich Bolz, Antonio Cuni, Maciej Fijalkowski, and Armin Rigo. 2009. Tracing the Meta-level: PyPy's Tracing JIT Compiler. In *Proceedings of the 4th workshop on the Implementation, Compilation, Optimization of Object-Oriented Languages and Programming Systems*. ACM, 18–25. https://doi.org/10.1145/1565824.1565827

[4] Jim des Rivières and John Wiegand. 2004. Eclipse: A platform for Integrating Development Tools. *IBM Systems Journal* 43, 2 (2004), 371–383. https://doi.org/10.1147/sj.432.0371

[5] Sebastian Erdweg, Tijs Van Der Storm, Markus Völter, Meinte Boersma, Remi Bosman, William R Cook, Albert Gerritsen, Angelo Hulshout, Steven Kelly, Alex Loh, et al. 2013. The State of the Art in Language Workbenches. In *International Conference on Software Language Engineering*. Springer, Springer International Publishing, 197–217. https://doi.org/10.1007/978-3-319-02654-1_11

[6] Moritz Eysholdt and Heiko Behrens. 2010. Xtext: Implement your Language Faster than the Quick and Dirty Way. In *Proceedings of the ACM International Conference on Object-Oriented Programming Systems Languages and Applications (OOPSLA) 2010*. ACM, 307–309. https://doi.org/10.1145/1869542.1869625

[7] Martin Fowler. 2010. *Domain-Specific Languages*. Pearson Education.

[8] Yoshihiko Futamura. 1999. Partial Evaluation of Computation Process–an Approach to a Compiler-Compiler. *Higher-Order and Symbolic Computation* 12, 4 (1999), 381–391. https://doi.org/10.1023/A:1010095604496

[9] Richard P Gabriel, Nickieben Bourbaki, Matthieu Devin, Patrick Dussud, David Gray, and Harlan Sexton. 1990. Foundation for a C++ Programming Environment. *Proceedings of C++ at Work* 90 (1990).

[10] Matthias Grimmer, Chris Seaton, Roland Schatz, Thomas Würthinger, and Hanspeter Mössenböck. 2015. High-performance Cross-language Interoperability in a Multi-language Runtime. In *ACM SIGPLAN Notices*, Vol. 51. ACM, 78–90. https://doi.org/10.1145/2936313.2816714

[11] Michael Haupt, Michael Perscheid, and Robert Hirschfeld. 2011. Type Harvesting: a Practical Approach to Obtaining Typing Information in Dynamic Programming Languages. In *Proceedings of the 2011 ACM Symposium on Applied Computing*. ACM, 1282–1289. https://doi.org/10.1145/1982185.1982464

[12] Reid Holmes and David Notkin. 2010. Enhancing Static Source Code Search with Dynamic Data. In *Proceedings of 2010 ICSE Workshop on Search-driven Development: Users, Infrastructure, Tools and Evaluation*. ACM, 13–16. https://doi.org/10.1145/1809175.1809179

[13] Dan Ingalls, Ted Kaehler, John Maloney, Scott Wallace, and Alan Kay. 1997. Back to the Future: the Story of Squeak, a Practical Smalltalk Written in Itself. In *SIGPLAN Notices*, Vol. 32. ACM, 318–326. https://doi.org/10.1145/263700.263754

[14] Lennart CL Kats and Eelco Visser. 2010. The Spoofax Language Workbench: Rules for Declarative Specification of Languages and IDEs. In *Proceedings of the ACM International Conference on Object-Oriented Programming Systems Languages and Applications (OOPSLA) 2010*, Vol. 45. ACM, 444–463. https://doi.org/10.1145/1869459.1869497

[15] Gregor Kiczales, Erik Hilsdale, Jim Hugunin, Mik Kersten, Jeffrey Palm, and William G Griswold. 2001. An Overview of AspectJ. In *European Conference on Object-Oriented Programming*. Springer, 327–354. https://doi.org/10.1007/3-540-45337-7_18

[16] Fabio Niephaus, Tim Felgentreff, and Robert Hirschfeld. 2018. GraalSqueak: A Fast Smalltalk Bytecode Interpreter Written in an AST Interpreter Framework. In *Proceedings of the 13th Workshop on Implementation, Compilation, Optimization of Object-Oriented Languages, Programs and Systems*. ACM, 30–35. https://doi.org/10.1145/3242947.3242948

[17] Fabio Niephaus, Tim Felgentreff, Tobias Pape, Robert Hirschfeld, and Marcel Taeumel. 2018. Live Multi-language Development and Runtime Environments. *The Art, Science, and Engineering of Programming* 2, 3 (mar 2018). https://doi.org/10.22152/programming-journal.org/2018/2/8

[18] Lukas Renggli, Tudor Gîrba, and Oscar Nierstrasz. 2010. Embedding Languages Without Breaking Tools. In *European Conference on Object-Oriented Programming*. Springer, 380–404. https://doi.org/10.1007/978-3-642-14107-2_19

[19] Manuel Rigger, Matthias Grimmer, Christian Wimmer, Thomas Würthinger, and Hanspeter Mössenböck. 2016. Bringing Low-level Languages to the JVM: Efficient Execution of LLVM IR on Truffle. In *Proceedings of the 8th International Workshop on Virtual Machines and Intermediate Languages*. ACM, 6–15. https://doi.org/10.1145/2998415.

[37] https://github.com/graalvm/graaljs (accessed 2018-12-17)

[38] https://github.com/oracle/truffleruby (accessed 2018-12-17)

2998416

[20] Armin Rigo and Samuele Pedroni. 2006. PyPy's Approach to Virtual Machine Construction. In *Companion to the 21st ACM SIGPLAN Symposium on Object-oriented Programming Systems, Languages, and Applications*. ACM, 944–953. https://doi.org/10.1145/1176617.1176753

[21] David Röthlisberger, Orla Greevy, and Oscar Nierstrasz. 2008. Exploiting Runtime Information in the IDE. In *Proceedings of the 16th IEEE International Conference on Program Comprehension, 2008. ICPC 2008.* IEEE, 63–72. https://doi.org/10.1109/ICPC.2008.32

[22] Éric Tanter, Jacques Noyé, Denis Caromel, and Pierre Cointe. 2003. Partial Behavioral Reflection: Spatial and Temporal Selection of Reification. *ACM SIGPLAN Notices* 38, 11 (2003), 27–46. https://doi.org/10.1145/949305.949309

[23] Michael Van De Vanter, Chris Seaton, Michael Haupt, Christian Humer, and Thomas Würthinger. 2018. Fast, Flexible, Polyglot Instrumentation Support for Debuggers and other Tools. *The Art, Science, and Engineering of Programming* 2, 3 (2018). https://doi.org/10.22152/programming-journal.org/2018/2/14

[24] Michael L Van De Vanter. 2015. Building Debuggers and other Tools: We Can Have it All. In *Proceedings of the 10th Workshop on Implementation, Compilation, Optimization of Object-Oriented Languages, Programs and Systems*. ACM, 2. https://doi.org/10.1145/2843915.2843917

[25] Guido van Rossum. 1995. *Python Tutorial.* techreport CS-R9526. Centrum voor Wiskunde en Informatica (CWI).

[26] Vlad Vergu, Pierre Neron, and Eelco Visser. 2015. DynSem: A DSL for Dynamic Semantics Specification. In *26th International Conference on Rewriting Techniques and Applications (RTA) 2015 (Leibniz International Proceedings in Informatics (LIPIcs))*, Maribel Fernández (Ed.), Vol. 36. Schloss Dagstuhl–Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 365–378. https://doi.org/10.4230/LIPIcs.RTA.2015.365

[27] Eelco Visser, Guido Wachsmuth, Andrew Tolmach, Pierre Neron, Vlad Vergu, Augusto Passalaqua, and Gabrieël Konat. 2014. A Language Designer's Workbench: A One-Stop-Shop for Implementation and Verification of Language Designs. In *Proceedings of the 2014 ACM International Symposium on New Ideas, New Paradigms, and Reflections on Programming & Software*. ACM, 95–111. https://doi.org/10.1145/2661136.2661149

[28] Markus Voelter and Vaclav Pech. 2012. Language Modularity with the MPS Language Workbench. In *Proceedings of the 34th International Conference on Software Engineering (ICSE), 2012.* IEEE, 1449–1450. https://doi.org/10.1109/ICSE.2012.6227070

[29] Christian Wimmer, Michael Haupt, Michael L Van De Vanter, Mick Jordan, Laurent Daynès, and Douglas Simon. 2013. Maxine: An Aproachable Virtual Machine for, and in, Java. *ACM Transactions on Architecture and Code Optimization (TACO)* 9, 4 (2013), 30. https://doi.org/10.1145/2400682.2400689

[30] Christian Wimmer and Thomas Würthinger. 2012. Truffle: A Self-optimizing Runtime System. In *Proceedings of the 3rd Annual Conference on Systems, Programming, and Applications: Software for Humanity*. ACM, 13–14. https://doi.org/10.1145/2384716.2384723

[31] Thomas Würthinger, Christian Wimmer, Christian Humer, Andreas Wöß, Lukas Stadler, Chris Seaton, Gilles Duboscq, Doug Simon, and Matthias Grimmer. 2017. Practical Partial Evaluation for High-performance Dynamic Language Runtimes. In *Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI) 2017*, Vol. 52. ACM, 662–676. https://doi.org/10.1145/3062341.3062381

[32] Thomas Würthinger, Christian Wimmer, Andreas Wöß, Lukas Stadler, Gilles Duboscq, Christian Humer, Gregor Richards, Doug Simon, and Mario Wolczko. 2013. One VM to Rule Them All. In *Proceedings of the 2013 ACM International Symposium on New Ideas, New Paradigms, and Reflections on Programming & Software*. ACM, 187–204. https://doi.org/10.1145/2509578.2509581

[33] Thomas Würthinger, Andreas Wöß, Lukas Stadler, Gilles Duboscq, Doug Simon, and Christian Wimmer. 2012. Self-optimizing AST Interpreters. In *ACM SIGPLAN Notices*, Vol. 48. ACM, 73–82. https://doi.org/10.1145/2384577.2384587